



Data Warehousing and
Business Intelligence
(IT3021)

Assignment 1

Registration Number – IT23246096

Name – D. D. Devanmith

Group – Y3.S2.WE.DS.01.01

Contents

Task 1: Data Set Selection	3
Task 2: Preparation of Data Sources	5
Task 3: Solution Architecture	19
Task 4: Data Warehouse Design & Development	24
Task 5: ETL Development	26
Task 6: ETL Development – Accumulating Fact Tables	44

Task 1: Data Set Selection

The dataset selected for this project is the **IPL Complete Dataset (2008-2024)**, available on Kaggle.

The IPL Complete Dataset offers a dynamic and data-rich scenario by focusing on real-world professional sports analytics, specifically the Indian Premier League. It captures comprehensive match details, team performances, and granular ball-by-ball activities, featuring complex relationships between players, teams, matches, and venues. The inclusion of detailed event data (like runs scored, extras, and wickets per delivery) across a long timespan adds depth, making it ideal for demonstrating ETL processes, data warehousing, and OLAP cube building in a realistic sports performance analytics environment.

The dataset consists of sufficient data as required by the assignment guidelines. It includes an extensive number of records, diverse attributes, and vast time coverage to demonstrate the extraction from multiple sources, ETL transformations, and comprehensive data warehouse design. It is primarily an OLTP (Online Transaction Processing) dataset. Some modifications were made to the original dataset derived from the source to prepare it for the ETL pipeline.

Original data set link:

<https://www.kaggle.com/datasets/patrickb1912/ipl-complete-dataset-20082020>

The IPL Complete Dataset contains two main interconnected files that reflect the complete operations and events of the tournament. Specifically, it includes:

- **Matches:** Contains the summary information of each match, including the match ID, participating teams, toss decisions, venue, match date, season, and the final result.
- **Deliveries:** A comprehensive ball-by-ball record of all matches. It describes the transactional events of the game, including the match ID, batting and bowling teams, innings, specific over and ball numbers, the batter, bowler, non-striker, runs scored by the batter, extra runs, and wicket details. *(Over 260,000 records)*

The match summary records (matches.csv) have dates spread over a vast period, covering all seasons from the inception of the tournament. (2008-2024).

The granular ball-by-ball data (deliveries.csv) mainly covers the operational events of the matches, and based on the dataset, the transactional records span continuously across the same extensive timeframe (2008-2024).

```

    erDiagram
        Delivery ||--o{ Player : Performs
        Delivery ||--o{ Match : Contains
        Delivery ||--o{ Match : Awards
        Delivery ||--o{ Team : Participates
        Team ||--o{ Match : Participates
        Venue ||--o{ Match : Hosts

        Delivery {
            string Dismissal_Kind
            string Noball_Runs
            string Inning
            string Legbye_Runs
            string Total_Runs
            string Bye_Runs
            string Wide_Runs
            string Extra_Runs
            string Is_Super_Over
            string Batsman_Runs
            string Ball_Num
            string Penalty_Runs
            string Over_Num
            string Delivery_ID PK
        }
        Player {
            string Player_ID PK
            string Player_Name
        }
        Team {
            string Team_ID PK
            string Team_Name
        }
        Match {
            string Match_ID PK
            string Season
            string Toss_Decision
            string Umpire
            string Toss_Decision
            string Win_by_runs
            string Result
            string DL_Applied
            string Match_Date
        }
        Venue {
            string Venue_ID PK
            string Venue_Name
            string City
        }
  
```

Task 2: Preparation of Data Sources

To meet the assignment requirement of utilizing at least two different types of data sources, the original IPL dataset—which was initially available purely as CSV files—was restructured and enriched. This reorganization allows for a clear separation of data types, enabling the demonstration of data extraction from heterogeneous sources, which aligns with standard data warehouse design principles.

Furthermore, to fulfill the requirement of enriching the data to include hierarchies, geographical data was introduced manually to complement the existing dataset.

The final data sources prepared include a SQL Server Database table, a CSV file, and an Excel file, containing the following information:

Source Type	Source Name	Description
SQL Database Table	Matches (in CricketSourceDB)	Contains match-level summary details (e.g., season, toss decisions, participating teams, and match winners). Originally from matches.csv.
CSV File	deliveries.csv	Contains granular, ball-by-ball transactional data representing the core operations of the matches (e.g., runs scored, wickets, extras).
Excel File	Venues.xlsx	A manually created file for data enrichment . It contains Venue_Name, City, and Country columns to establish a geographic hierarchy (Country -> City -> Stadium) that was not available in the original raw dataset.

Detailed Attribute Mapping for Heterogeneous Sources

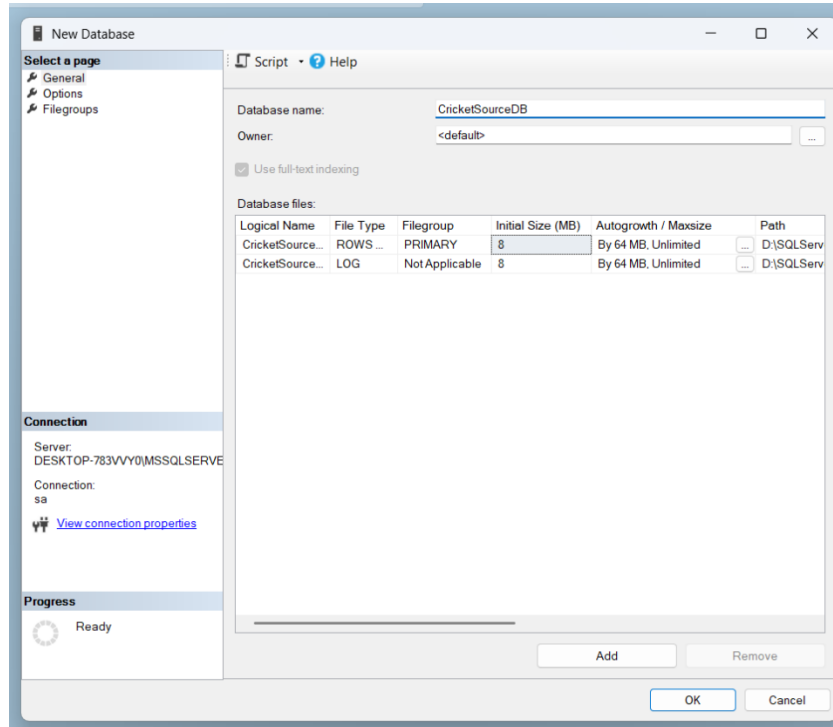
To ensure a structured and seamless ETL process, the attributes from each source were mapped based on their functional role in the data warehouse:

- **Match Summary Data (SQL Server Source):** The Matches table in the SQL database serves as the primary source for match-level information. Key attributes such as id, season, city, and date provide the temporal and spatial context, while fields like team1, team2, toss_winner, and winner define the participating entities and match outcomes.
- **Ball-by-Ball Transactional Data (CSV Source):** The deliveries.csv file provides the most granular level of data. Attributes like match_id act as the linking key to the match summary. Performance metrics such as batsman_runs, extra_runs, and total_runs are mapped as the primary facts, while categorical fields like batsman, bowler, and batting_team provide the basis for dimensional analysis.
- **Geographical Hierarchy Enrichment (Excel Source):** The Venues.xlsx file was specifically designed to bridge the gap in the original dataset's geographical data. By mapping the Venue_Name from the source to the City and the newly introduced Country attribute, a clear data hierarchy (**Country -> City -> Venue**) is established. This mapping is essential for creating the drill-down levels required in the SSAS OLAP Cube.

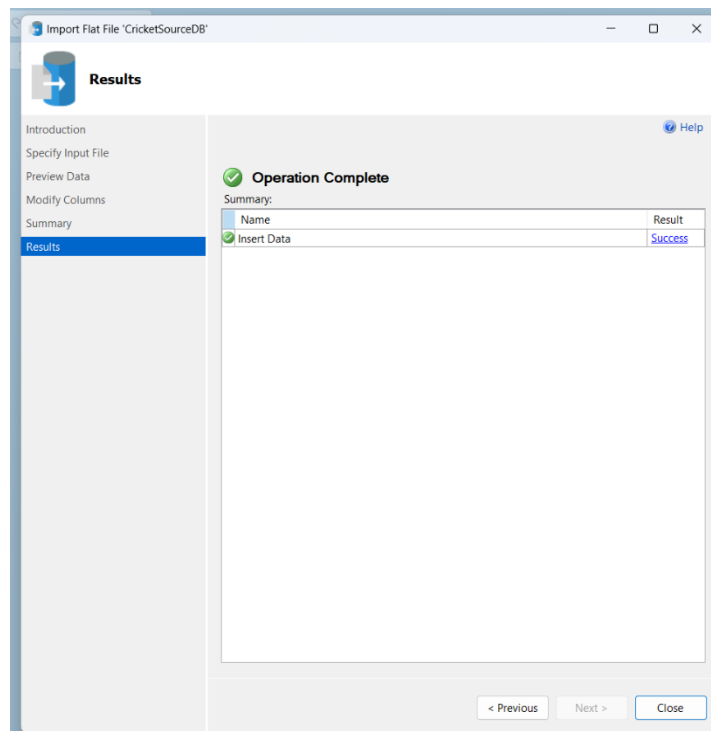
Mapping Summary for ETL Implementation

In the SSIS pipeline, these attributes are mapped to their respective staging tables (Stg_Matches, Stg_Deliveries) and eventually to the Dimensional Model (Cricket_DW). The match_id serves as the critical join key between the CSV and SQL sources, while the Venue_Name acts as the lookup key between the SQL data and the Excel enrichment file.

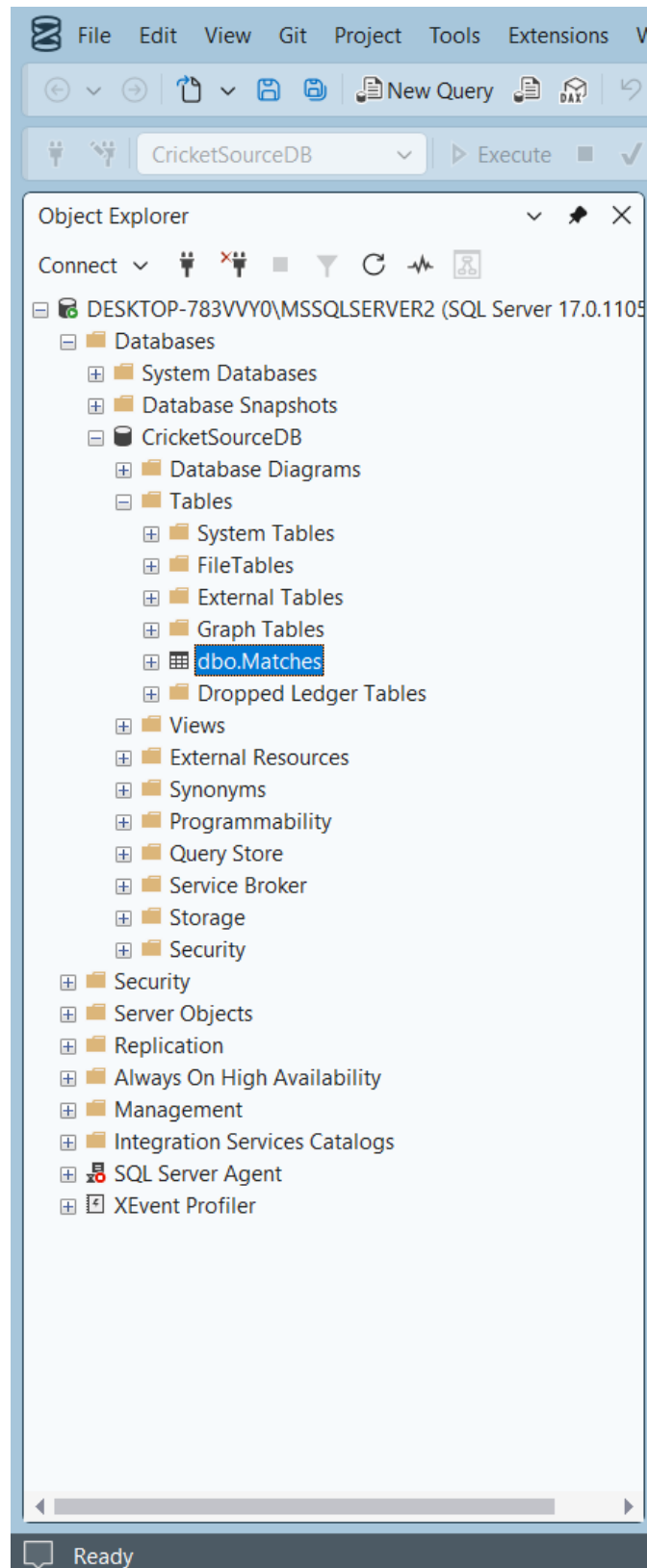
Database Creation (CricketSourceDB)



Import Wizard Setup for the Matches Table



Tables Listed in Object Explorer (CricketSourceDB)



Query Result for Sample Data from the Database

Object Explorer: Connect to DESKTOP-783VVY0\MSSQLSERVER2 (SQL Server 17.0.11020.1) > Databases > CricketSourceDB > Tables > dbo.Match

SQLQuery1.sql - eDB (sa (70))

```
1 SELECT TOP (1000) [id]
2
3 [season]
4
5 [city]
6
7 [date]
8
9 [match_type]
10
11 [player_of_match]
12
13 [venue]
14
15 [team1]
16
17 [team2]
18
19 [toss_winner]
20
21 [toss_decision]
22
23 [winner]
```

Results: Messages

id	season	city	date	match_type	player_of_match	venue	team1	team2	toss_winner	toss_decision	winner
1	335982	200708	Bangalore	2008-04-18	League	BB McCullum	M Chinnaswamy Stadium	Royal Challengers Bangalore	Kolkata Knight Riders	Royal Challengers Bangalore	field
2	335983	200708	Chandigarh	2008-04-19	League	MEK Hussey	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Chennai Super Kings	Chennai Super Kings	bat
3	335984	200708	Dehi	2008-04-19	League	MF Maharoof	Feroz Shah Kotla	Dehi Daredevils	Rajasthan Royals	bat	Dehi Dare
4	335985	200708	Mumbai	2008-04-20	League	MI Boucher	Wankhede Stadium	Mumbai Indians	Royal Challengers Bangalore	bat	Royal Cha
5	335986	200708	Kolkata	2008-04-20	League	DJ Hussey	Eden Gardens	Kolkata Knight Riders	Deccan Chargers	Deccan Chargers	bat
6	335987	200708	Jalpur	2008-04-21	League	SRI Watson	Sawai Mansingh Stadium	Rajasthan Royals	Kings XI Punjab	Kings XI Punjab	bat
7	335988	200708	Hyderabad	2008-04-22	League	V Sehwag	Rajiv Gandhi International Stadium, Uppal	Deccan Chargers	Dehi Daredevils	Deccan Chargers	bat
8	335989	200708	Chennai	2008-04-23	League	ML Hayden	MA Chidambaram Stadium, Chepauk	Chennai Super Kings	Mumbai Indians	field	Chennai Su
9	335990	200708	Hyderabad	2008-04-24	League	SM Katich	Rajiv Gandhi International Stadium, Uppal	Deccan Chargers	Rajasthan Royals	field	Rajasthan
10	335991	200708	Chandigarh	2008-04-25	League	KC Sangakkara	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Mumbai Indians	field	Kings XI P
11	335992	200708	Bangalore	2008-04-26	League	SRI Watson	M Chinnaswamy Stadium	Royal Challengers Bangalore	Rajasthan Royals	Rajasthan Royals	field
12	335993	200708	Chennai	2008-04-26	League	JDP Oram	MA Chidambaram Stadium, Chepauk	Chennai Super Kings	Kolkata Knight Riders	bat	Chennai Su
13	335994	200708	Mumbai	2008-04-27	League	AC Gilchrist	Dr DY Patil Sports Academy	Mumbai Indians	Deccan Chargers	field	Deccan Ch
14	335995	200708	Chandigarh	2008-04-27	League	SM Katich	Punjab Cricket Association Stadium, Mohali	Kings XI Punjab	Dehi Daredevils	bat	Kings XI P
15	335996	200708	Bangalore	2008-04-28	League	MS Dhoni	M Chinnaswamy Stadium	Royal Challengers Bangalore	Chennai Super Kings	Chennai Super Kings	bat
16	335997	200708	Kolkata	2008-04-29	League	ST Jayasuriya	Eden Gardens	Kolkata Knight Riders	Mumbai Indians	Kolkata Knight Riders	bat
17	335998	200708	Dehi	2008-04-30	League	GD McGrath	Feroz Shah Kotla	Dehi Daredevils	Royal Challengers Bangalore	Royal Challengers Bangalore	field

Query executed successfully. DESKTOP-783VVY0\MSSQLSERVER... sa (70) CricketSourceDB 00:00:00 Row: 1, Col: 1 1,000 rows

Flat Files Directory (CSV and Excel Sources)

IT23246096 • Practice 2 code: IT23246096_Prac_ Data Types • IT23246096_Prac_ IT23246096 • IT23246096_Prac_ varrays, ne: • app.log IT23246096_Prac_ Task 2 Data • deliveries. x + -

File Edit View

```
match_id,inning,batting_team,bowling_team,over,ball,batter,bowler,non_striker,batsman_runs,total_runs,extras_type,is_wicket,player_dismissed,dismissal_kind,fielder
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,1,SC Ganguly,P Kumar,BB McCullum,0,1,legbyes,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,2,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,3,BB McCullum,P Kumar,SC Ganguly,0,1,1,wides,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,4,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,5,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,6,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,0,7,BB McCullum,P Kumar,SC Ganguly,0,1,1,legbyes,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,1,BB McCullum,Z Khan,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,2,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,3,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,4,BB McCullum,Z Khan,SC Ganguly,6,0,6,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,5,BB McCullum,Z Khan,SC Ganguly,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,1,6,BB McCullum,Z Khan,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,1,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,2,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,3,SC Ganguly,P Kumar,BB McCullum,0,1,1,legbyes,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,4,BB McCullum,P Kumar,SC Ganguly,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,5,BB McCullum,P Kumar,SC Ganguly,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,2,6,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,2,BB McCullum,AA Noffke,SC Ganguly,0,5,5,wides,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,3,BB McCullum,AA Noffke,SC Ganguly,0,1,1,legbyes,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,4,SC Ganguly,AA Noffke,BB McCullum,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,5,SC Ganguly,AA Noffke,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,6,SC Ganguly,AA Noffke,BB McCullum,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,3,7,BB McCullum,AA Noffke,SC Ganguly,6,0,6,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,1,SC Ganguly,P Kumar,BB McCullum,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,2,SC Ganguly,P Kumar,BB McCullum,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,3,BB McCullum,P Kumar,SC Ganguly,4,0,4,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,4,BB McCullum,P Kumar,SC Ganguly,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,5,BB McCullum,P Kumar,SC Ganguly,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,4,6,SC Ganguly,P Kumar,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,1,BB McCullum,Z Khan,SC Ganguly,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,2,SC Ganguly,Z Khan,BB McCullum,0,0,0,1,SC Ganguly,caught,JH Kallis
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,3,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,4,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,5,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,5,6,RT Ponting,Z Khan,BB McCullum,0,0,0,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,1,BB McCullum,AA Noffke,RT Ponting,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,2,RT Ponting,AA Noffke,BB McCullum,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,3,BB McCullum,AA Noffke,RT Ponting,1,0,1,0,NA,NA,NA
335982,1,Kolkata Knight Riders,Royal Challengers Bangalore,6,4,RT Ponting,AA Noffke,BB McCullum,2,0,2,0,NA,NA,NA
```

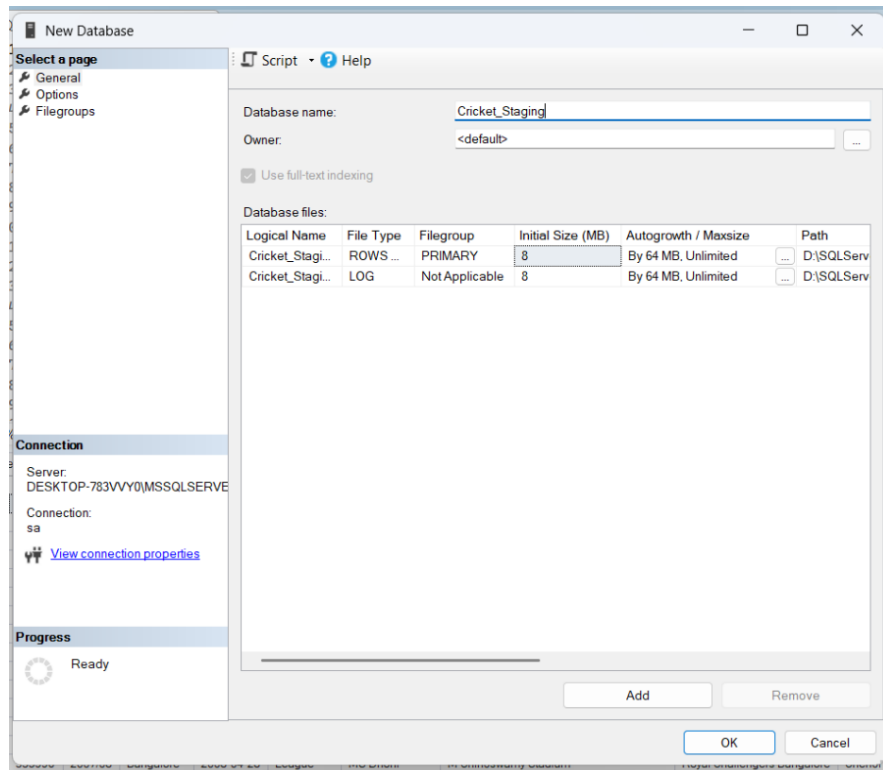
Ln 1, Col 1 26,759,032 characters Plain text 100% Windows (CRLF) UTF-8

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	venue	city	country														
2	Arun Jaitley Stadium	Delhi	India														
3	Arun Jaitley Stadium, Delhi	Delhi	India														
4	Barabati Stadium	Cuttack	India														
5	Barsapara Cricket Stadium, Guwahati	Guwahati	India														
6	Bharat Ratna Shri Atal Bihari Vajpayee Ekana Cricket Stadium, Lucknow	Lucknow	India														
7	Brabourne Stadium	Mumbai	India														
8	Brabourne Stadium, Mumbai	Mumbai	India														
9	Buffalo Park	East London	South Africa														
10	De Beers Diamond Oval	Kimberley	South Africa														
11	Dr DY Patil Sports Academy	Mumbai	India														
12	Dr DY Patil Sports Academy, Mumbai	Mumbai	India														
13	Dr DY Patil Sports Academy, Mumbai	Navi Mumbai	India														
14	Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket Stadium	Visakhapatnam	India														
15	Dr. Y.S. Rajasekhara Reddy ACA-VDCA Cricket Stadium, Visakhapatnam	Visakhapatnam	India														
16	Dubai International Cricket Stadium	Dubai	UAE														
17	Dubai International Cricket Stadium	MA	UAE														
18	Eden Gardens	Kolkata	India														
19	Eden Gardens, Kolkata	Kolkata	India														
20	Feroz Shah Kotla	Delhi	India														
21	Green Park	Kanpur	India														
22	Himachal Pradesh Cricket Association Stadium	Dharamsala	India														
23	Himachal Pradesh Cricket Association Stadium, Dharamsala	Dharamsala	India														
24	Holkar Cricket Stadium	Indore	India														
25	JSCA International Stadium Complex	Ranchi	India														
26	Kingsmead	Durban	South Africa														
27	M Chinnasswamy Stadium	Bangalore	India														
28	M Chinnasswamy Stadium, Bengaluru	Bengaluru	India														
29	M Chinnasswamy Stadium	Bengaluru	India														
30	MA Chidambaram Stadium	Chennai	India														

Create the Staging Database

A staging database is used to temporarily store and prepare data before loading it into the final Data Warehouse. It acts as a buffer area where raw data from multiple heterogeneous sources (our SQL Source DB, Excel, and CSV files) is collected, cleaned, transformed, and validated without affecting the source systems.

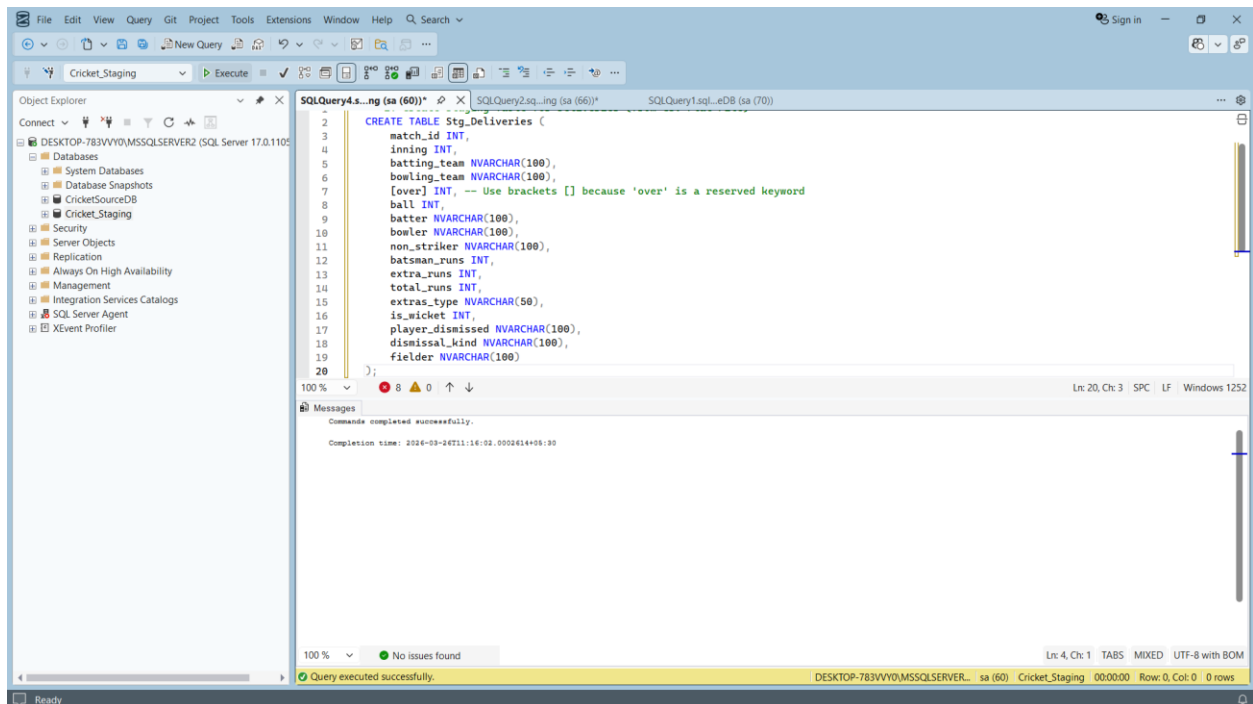
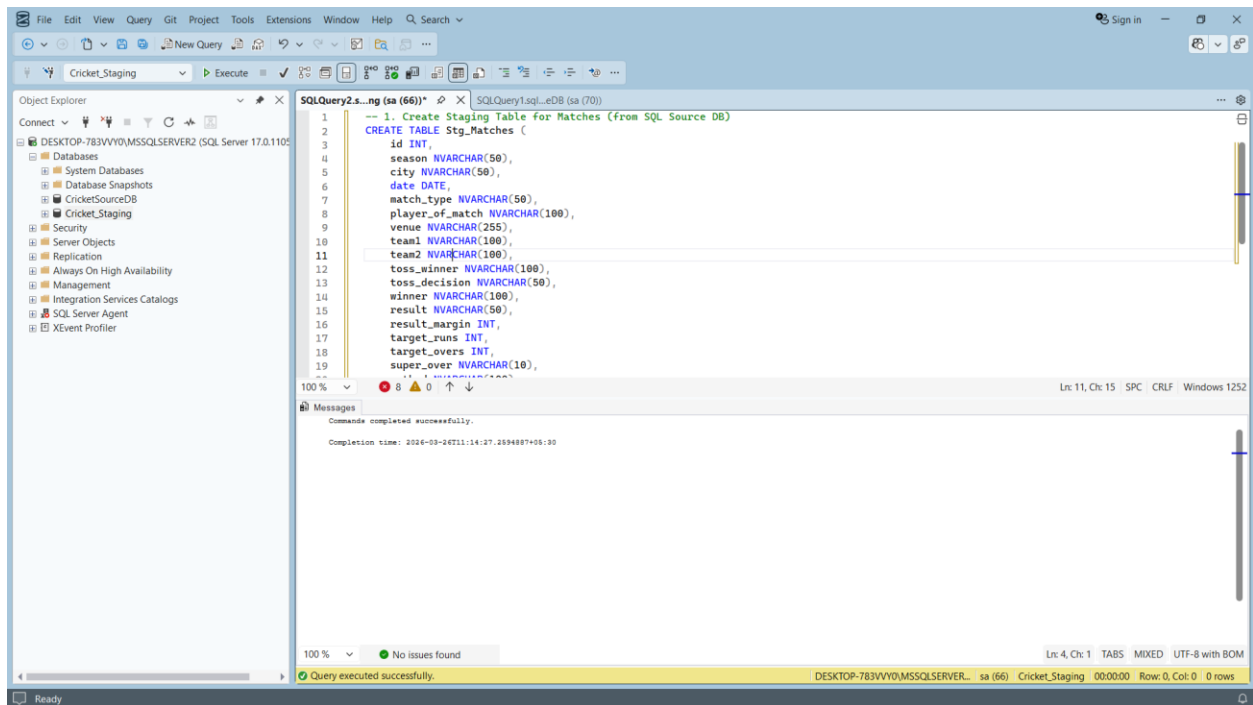
For this project, a staging database named **Cricket_Staging** was created.

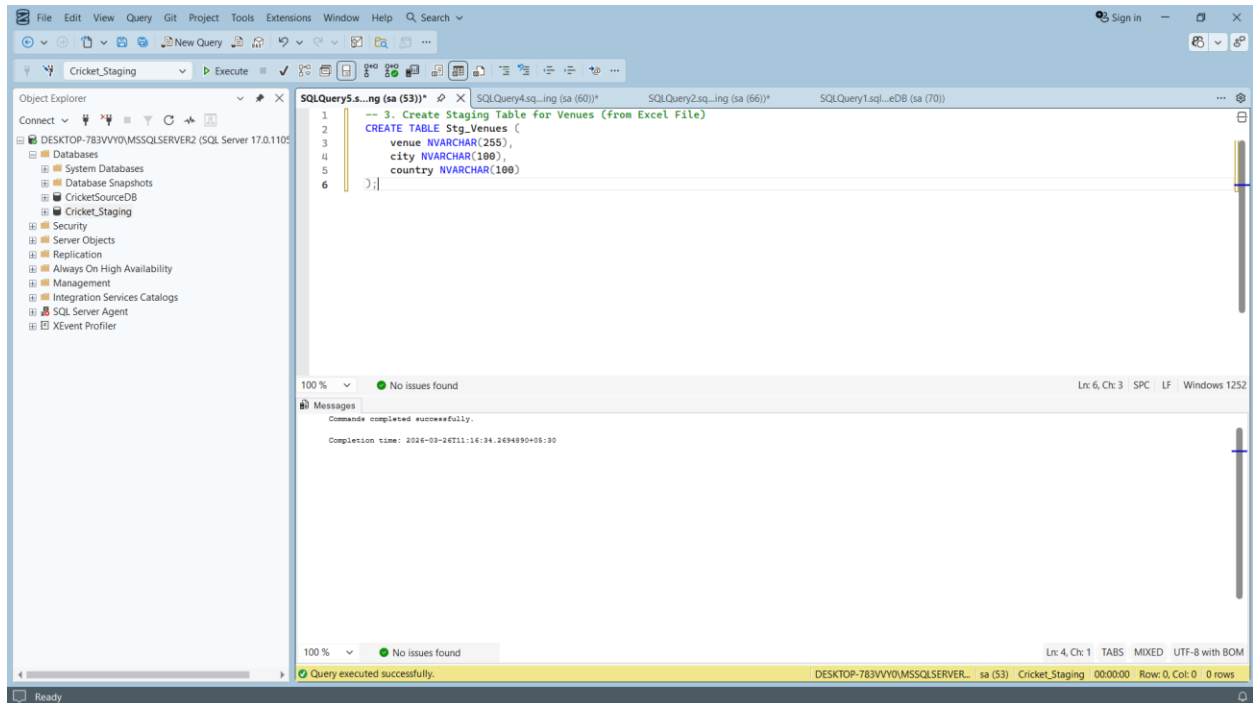


Create Staging Tables (Cricket_Staging)

Next, we created tables inside the Cricket_Staging database that match the structure of our source files. All data types were set to appropriately handle raw, uncleaned data (e.g., using VARCHAR for most fields initially).

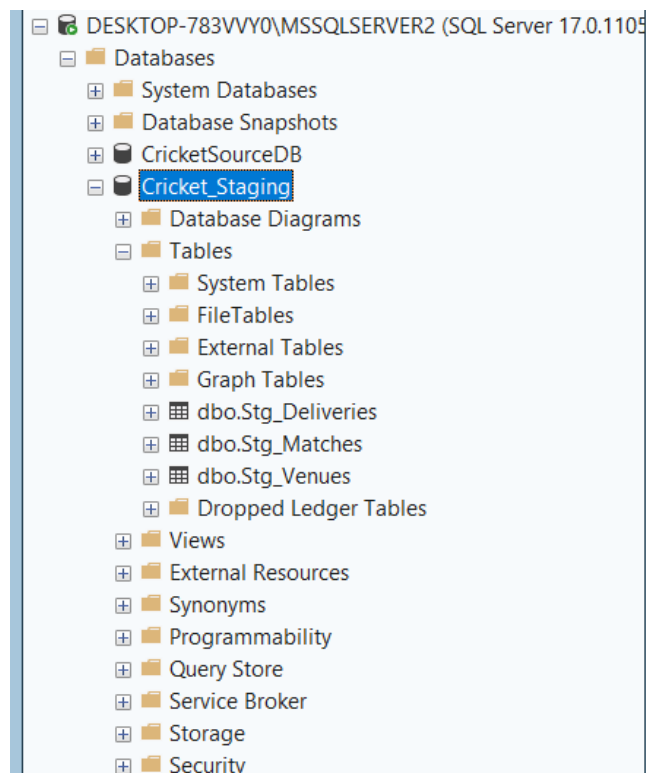
- **Stg_Matches:** To hold the match summary and venue data.
- **Stg_Deliveries:** To hold the massive ball-by-ball transactional data.
- **Stg_Venues:** To hold the enriched venue and geographic data extracted from the Excel file.





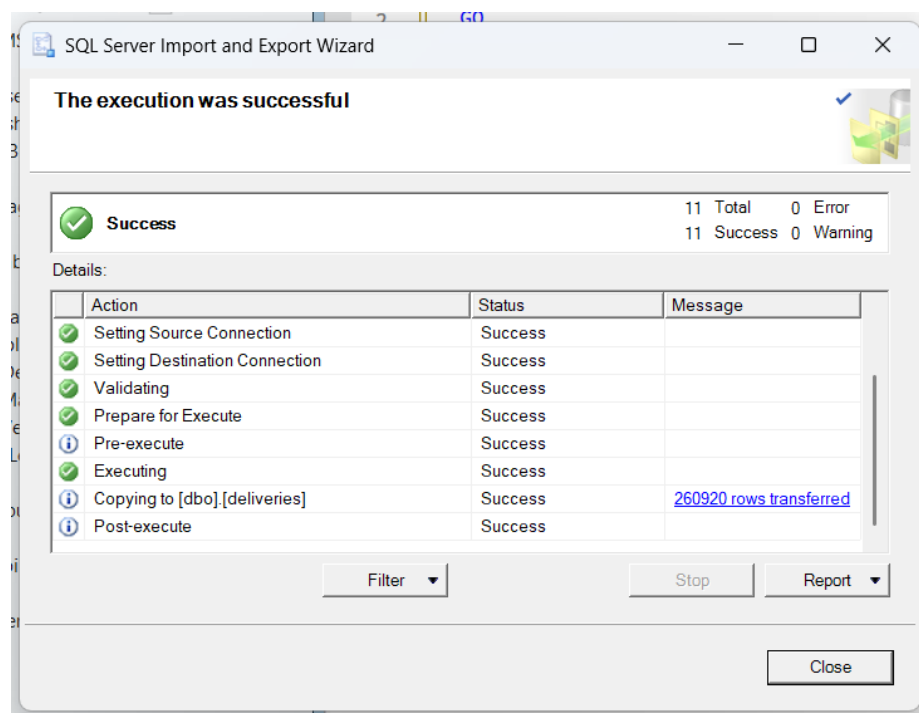
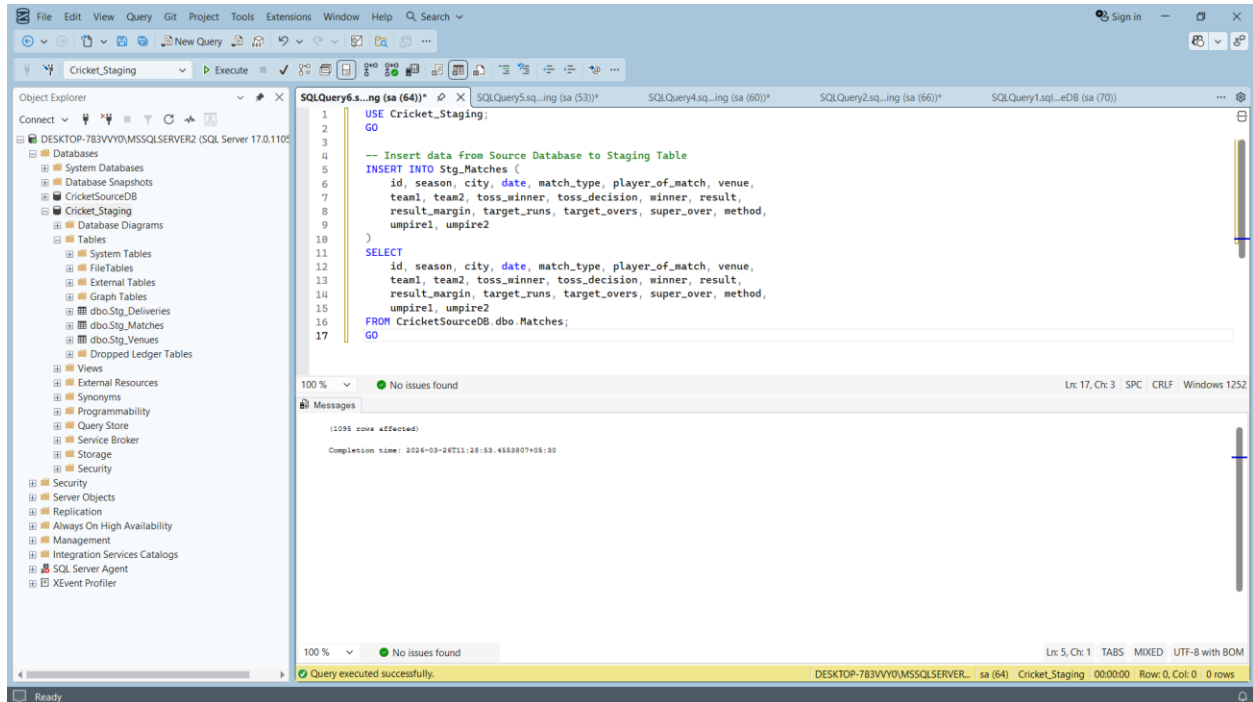
Tables of the Staging Database

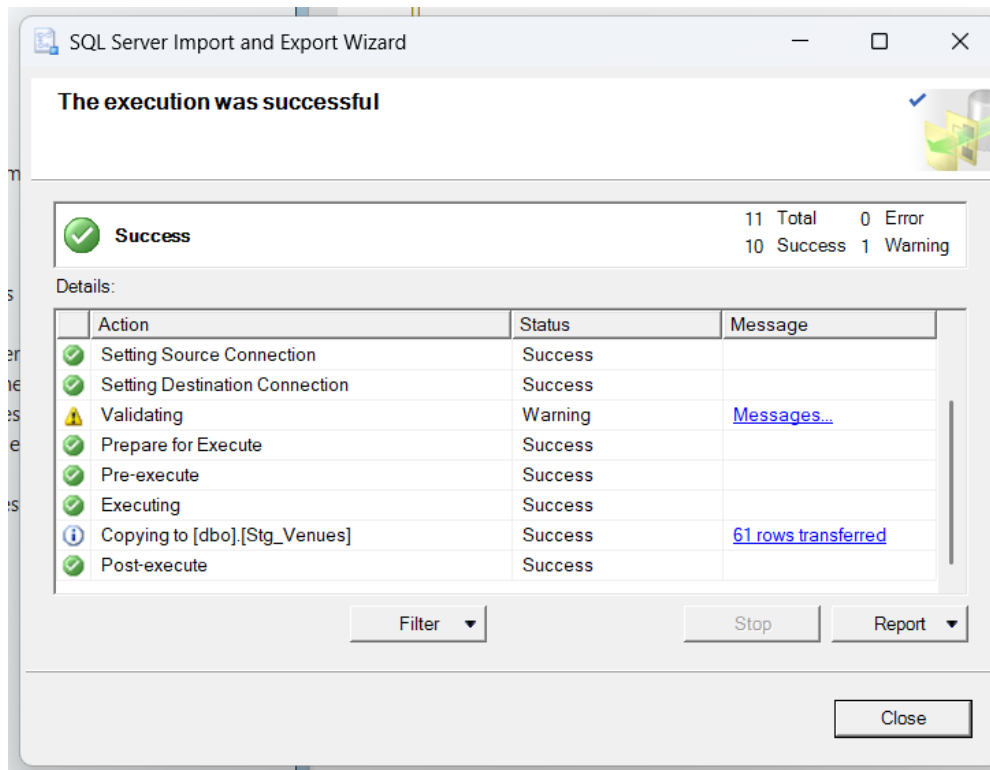
Once the tables were created successfully, they were verified in the Object Explorer.



Transfer Data from Source to Staging

The data from our initial sources (CricketSourceDB and flat files) was extracted and loaded into the Cricket_Staging database. This was achieved using the SQL Server Import/Export Wizard (and/or simple SSIS data flow tasks) to ensure raw data was quickly moved into the staging area for further transformation.





Verify the Data Was Moved

To ensure data integrity and confirm that no data was lost during the extraction to the staging area, row counts were verified between the source files/databases and the newly loaded staging tables. The counts matched perfectly.

Verifying Matches Data:

-- Count in Staging

```
SELECT COUNT(*) FROM Cricket_Staging.dbo.Stg_Matches;
```

-- Count in Source Database

```
SELECT COUNT(*) FROM CricketSourceDB.dbo.Matches;
```

Verifying Deliveries Data:

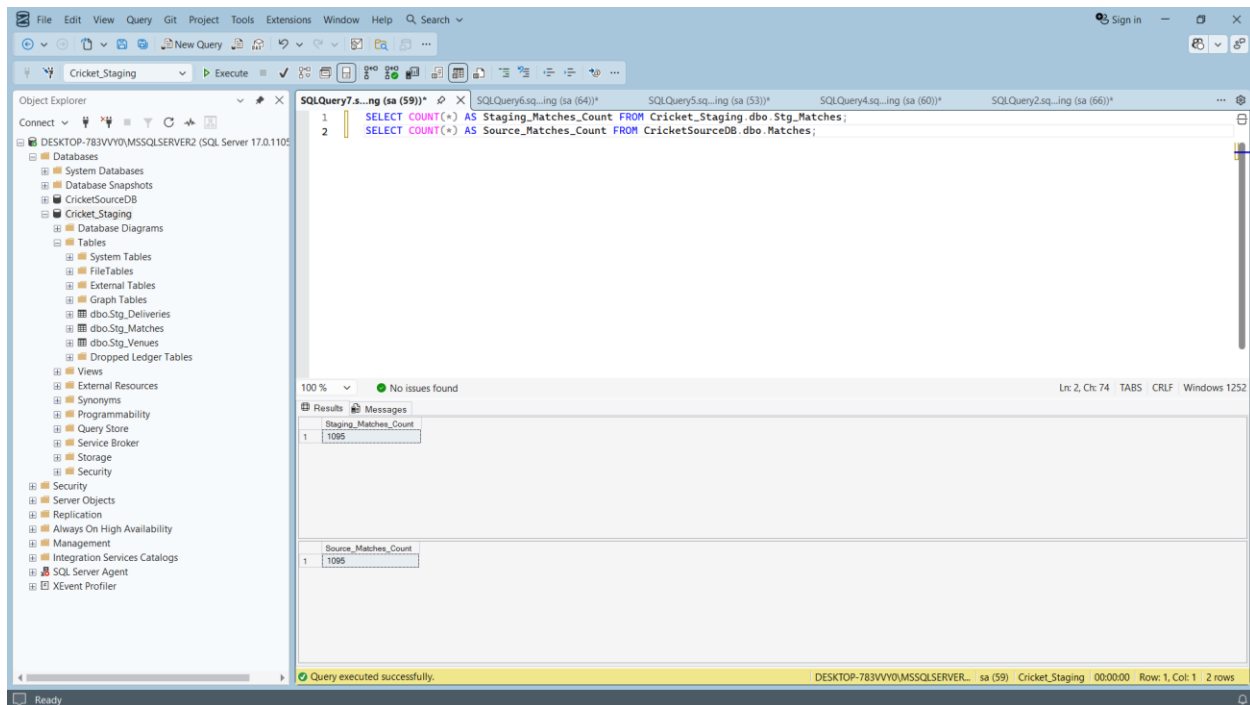
-- Count in Staging (Compared against the known row count of the raw deliveries.csv file)

```
SELECT COUNT(*) FROM Cricket_Staging.dbo.Stg_Deliveries;
```

Verifying Venues Data:

-- Count in Staging (Compared against the rows in the Venues Excel file)

```
SELECT COUNT(*) FROM Cricket_Staging.dbo.Stg_Venues;
```



Object Explorer

Connect

DESKTOP-783VVY0\MSSQLSERVER2 (SQL Server 17.0.1105)

Databases

- System Databases
- Database Snapshots
- CricketSourceDB
- Cricket_Staging
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.Stg_Deliveries
 - dbo.Stg_Matches
 - dbo.Stg_Venues
 - Dropped Ledger Tables
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
- Security
 - Server Objects
 - Replication
 - Always On High Availability
 - Management
 - Integration Services Catalogs
 - SQL Server Agent
 - XEvent Profiler

SQLQuery7.sql...ing (sa (59))*

```
1 | SELECT COUNT(*) FROM Cricket_Staging.dbo.Stg_Deliveries;
```

100 % No issues found Lrn 1, Ch: 57 TABS CRLF Windows 1252

Results Messages

	(No column name)
1	260820

Query executed successfully. DESKTOP-783VVY0\MSSQLSERVER... sa (59) Cricket_Staging 00:00:00 Row: 1, Col: 1 1 rows

Object Explorer

Connect

DESKTOP-783VVY0\MSSQLSERVER2 (SQL Server 17.0.1105)

Databases

- System Databases
- Database Snapshots
- CricketSourceDB
- Cricket_Staging
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.Stg_Deliveries
 - dbo.Stg_Matches
 - dbo.Stg_Venues
 - Dropped Ledger Tables
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
- Security
 - Server Objects
 - Replication
 - Always On High Availability
 - Management
 - Integration Services Catalogs
 - SQL Server Agent
 - XEvent Profiler

SQLQuery7.sql...ing (sa (59))*

```
1 | SELECT COUNT(*) AS Staging_Venues_Count FROM Cricket_Staging.dbo.Stg_Venues;
```

100 % No issues found Lrn 1, Ch: 77 TABS CRLF Windows 1252

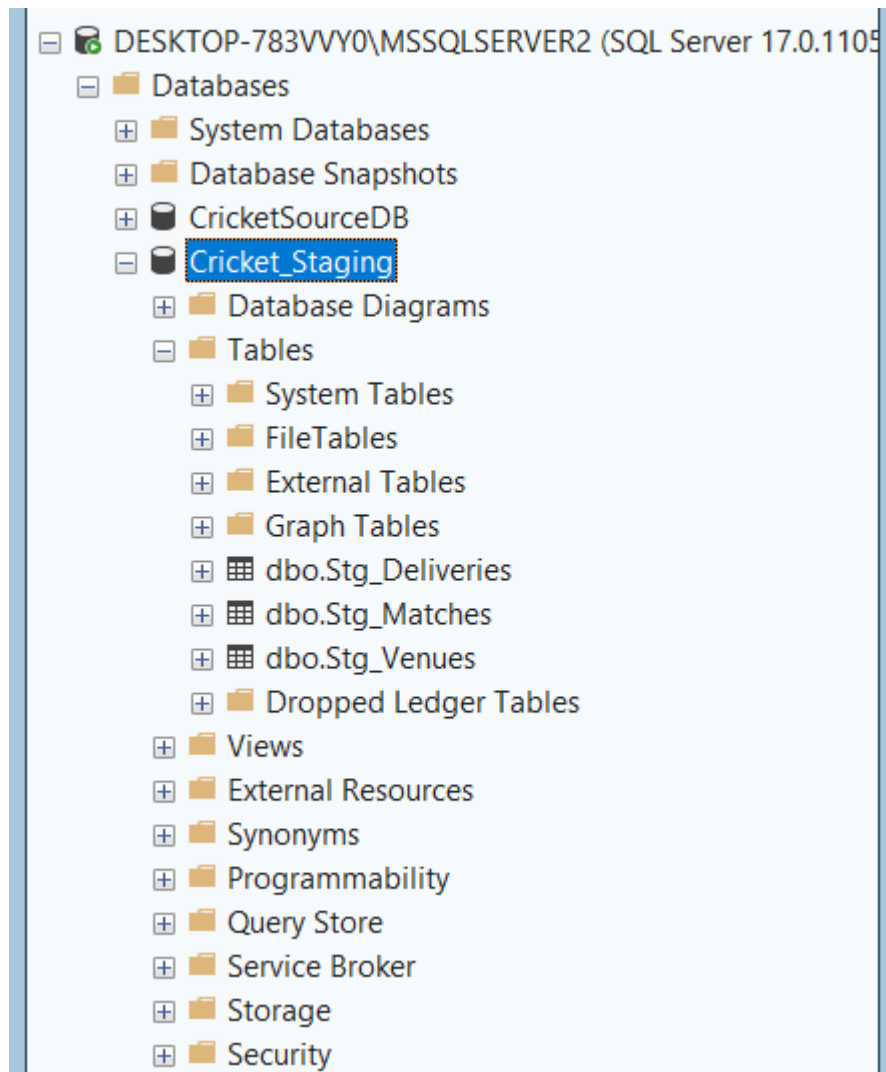
Results Messages

	Staging_Venues_Count
1	61

Query executed successfully. DESKTOP-783VVY0\MSSQLSERVER... sa (59) Cricket_Staging 00:00:00 Row: 1, Col: 1 1 rows

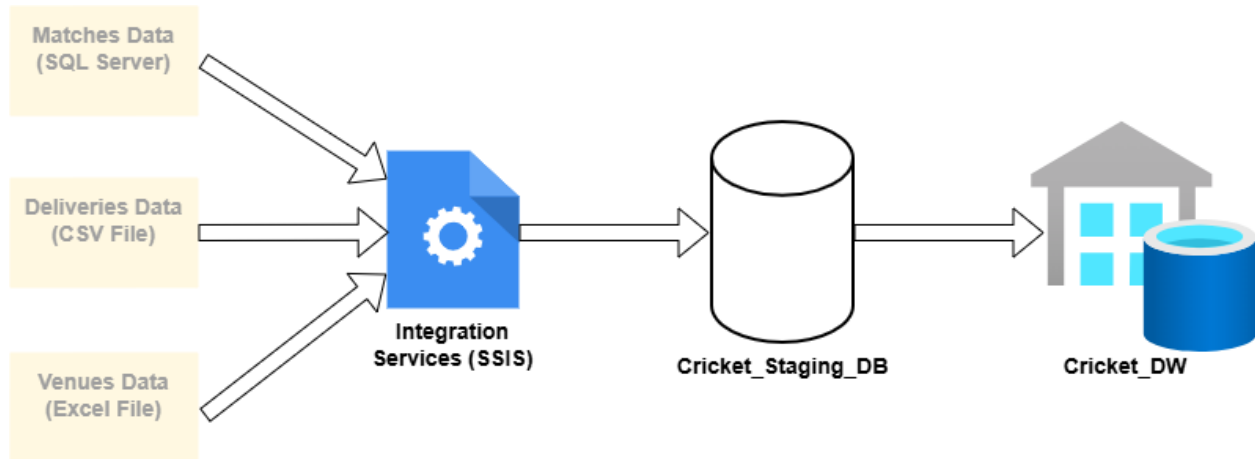
Final Output

Then the tables were loaded to the “Cricket_Staging” database.

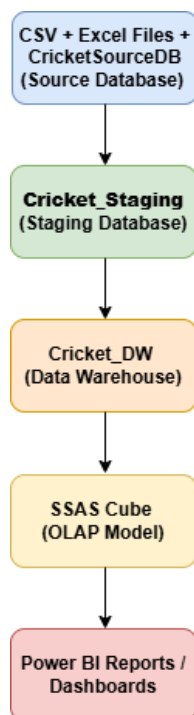


Task 3: Solution Architecture

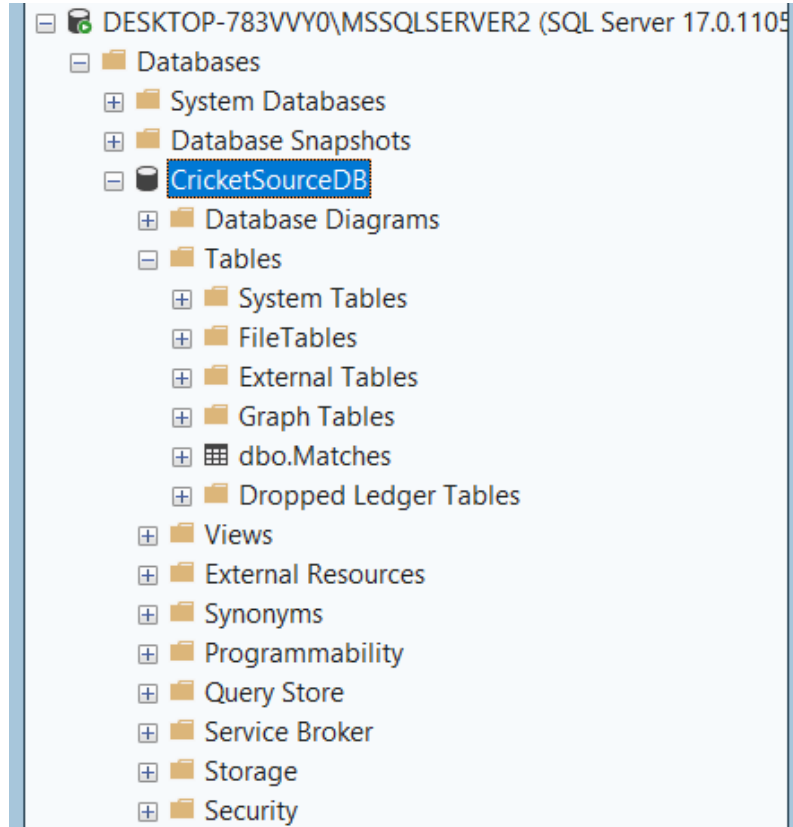
The below figure shows the high-level diagram for the Data Warehousing process (ETL Architecture).



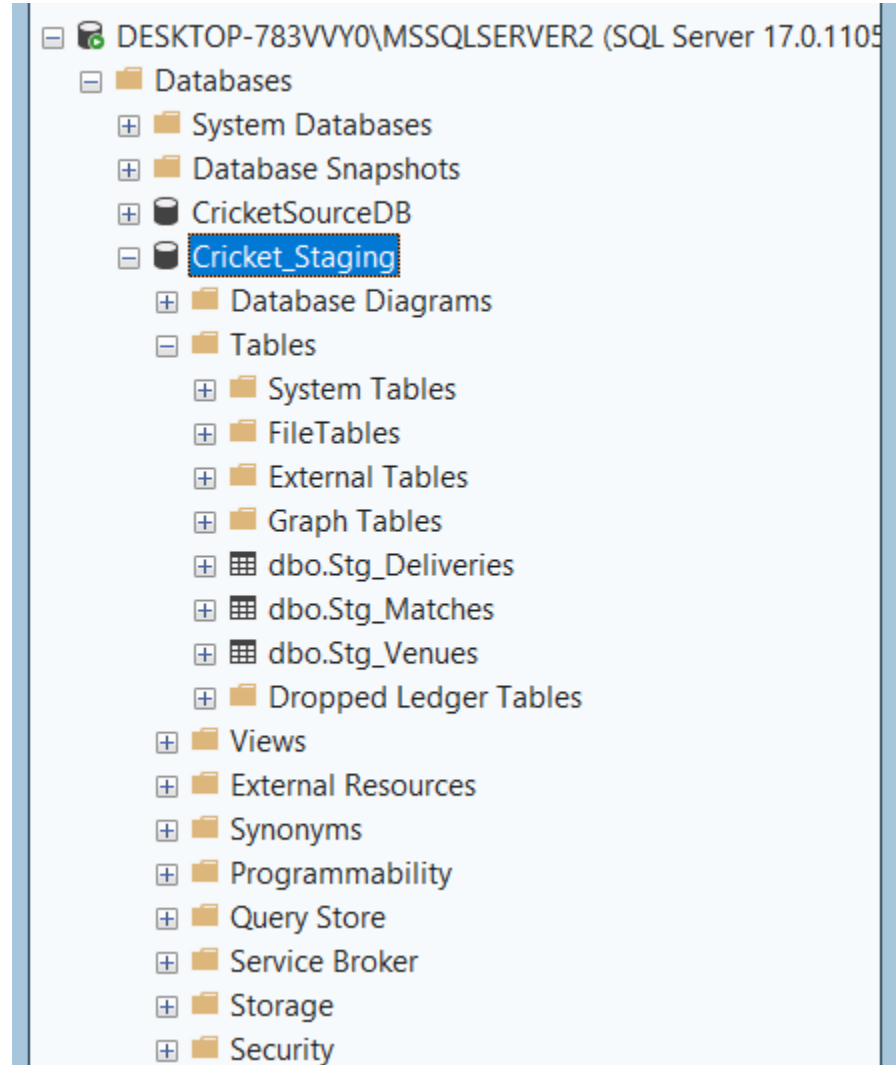
As illustrated in the figure, the data originates from heterogeneous source systems, including a SQL database (CricketSourceDB), a CSV flat file (deliveries.csv), and an enriched Excel file (Venues.xlsx). These raw data sources are extracted and transferred into a staging area (Cricket_Staging) using SQL Server Integration Services (SSIS) and Import processes. The staging database acts as a temporary buffer and intermediate storage location where the raw data is housed, cleansed, transformed, and prepared before being loaded into the final dimensional Data Warehouse (Cricket_DW).



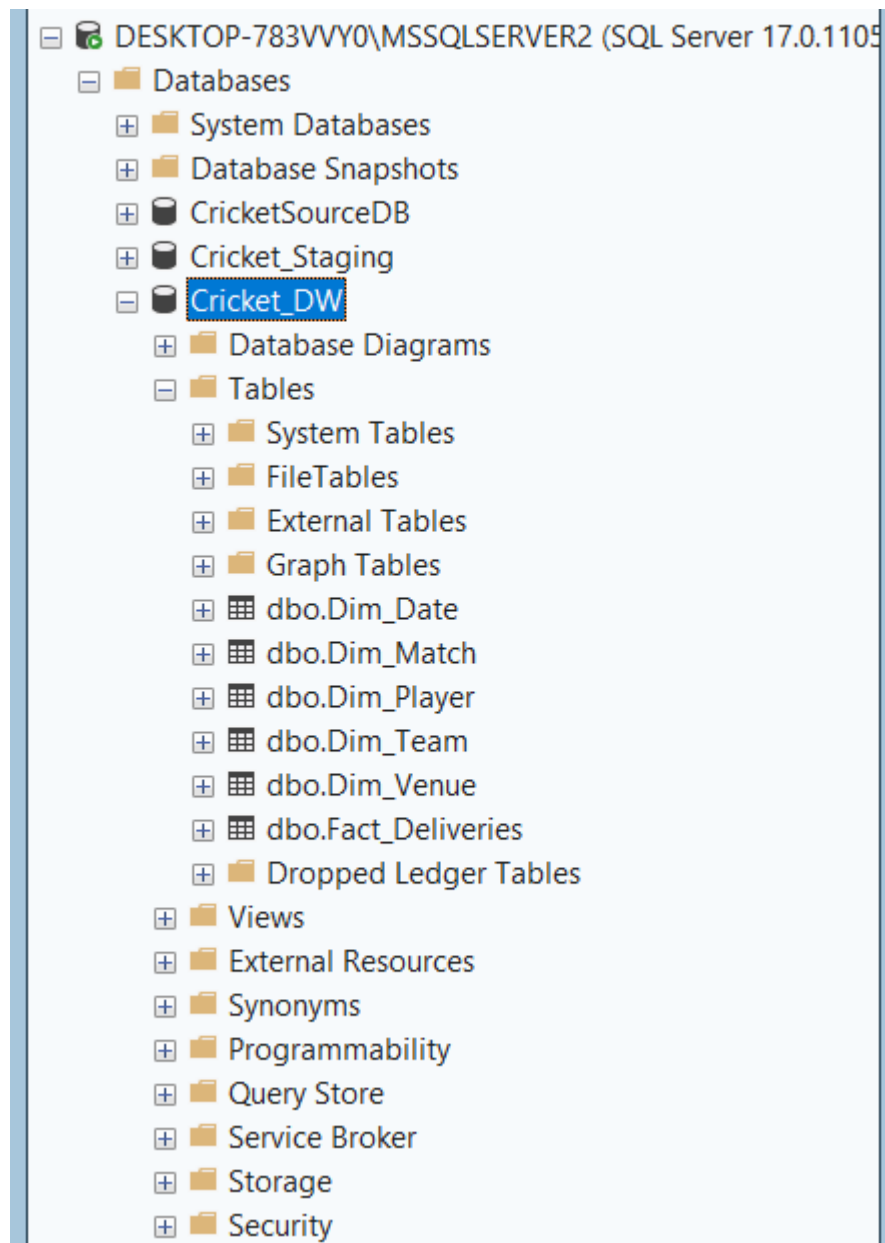
Database- CricketSourceDB



Staging Database- Cricket_Staging



Data Warehouse- Cricket_DW(Data was Transformed and Loaded to Data Warehouse)



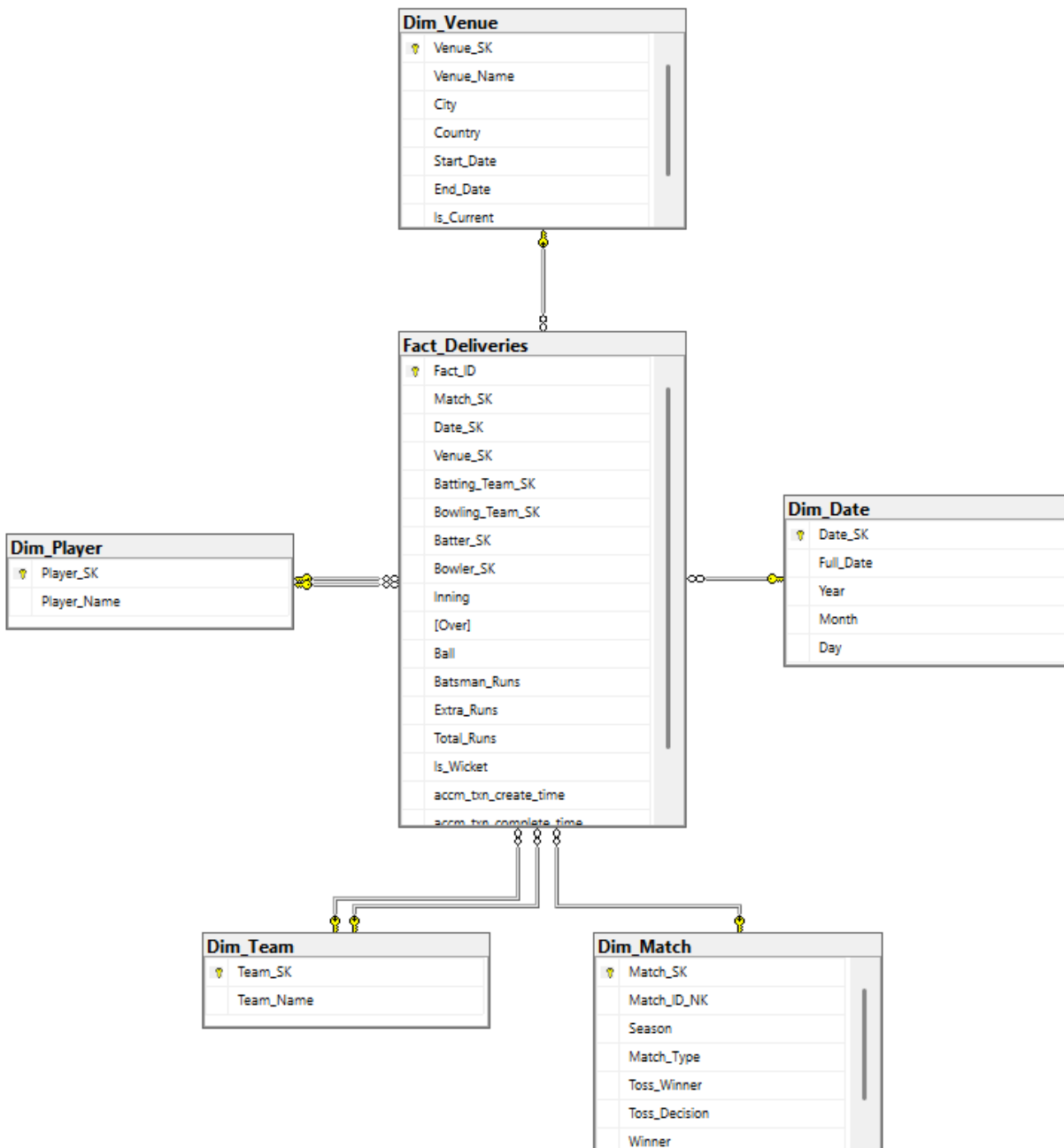
The Data Warehouse and Business Intelligence (DW & BI) solution starts with raw cricket data captured across multiple formats (CSV, Excel, and SQL Database). The data is then extracted and moved into a staging database (Cricket_Staging) where data profiling, cleansing, and necessary transformations occur (such as handling missing values and surrogate key lookups). The cleansed data is then loaded into the main data warehouse

(Cricket_DW), which is modeled in a Star Schema format consisting of central fact tables and surrounding dimension tables. From this data warehouse, SSAS cubes will be built to organize the data into hierarchies and aggregates for high-speed analysis. Finally, Power BI will be used to create interactive reports and dashboards, allowing users to perform in-depth analytics and visualize match and player performances.

Summarized Descriptions of Each Component

Component	Description
Heterogeneous Sources (CSV & Excel)	The original raw files containing granular ball-by-ball transactional data (deliveries.csv) and enriched geographical venue data (Venues.xlsx).
CricketSourceDB (Source Database)	A SQL database where the extracted raw match summary data (matches.csv) is initially loaded. It acts as the primary relational source system.
Cricket_Staging (Staging Database)	Temporary storage buffer used for extracting data from all sources. It is utilized to cleanse, validate, and transform data (e.g., resolving duplicate keys and missing values) before moving it into the data warehouse.
Cricket_DW (Data Warehouse)	The main analytical database designed using a Star Schema. It contains dimensional tables (Dim_Player, Dim_Venue, etc.) and a central fact table (Fact_Deliveries) optimized for fast analytical querying.
SSAS Cube (OLAP Model)	An analytical model built using SQL Server Analysis Services (SSAS), creating multidimensional cubes with hierarchies, measures (runs, wickets), and aggregations for fast data slicing.
Power BI Reports / Dashboards	The visualization layer where end-users interact with the processed data through dashboards, charts, and drill-through reports connected to the OLAP cubes.

Task 4: Data Warehouse Design & Development



The key icon should appear on the dimension table side (indicating the Primary Surrogate Key), and the infinity (∞) symbol should appear on the fact table side (indicating the Foreign Key).

- **Grain Definition:** The grain of the fact table is explicitly defined as **one row per single ball bowled in an innings**.

- One central fact table, **Fact_Deliveries** (which stores the granular ball-by-ball transactional metrics such as runs and wickets), was created during the warehouse design.
- Therefore, a Star Schema was used, where a central fact table is directly surrounded by its associated dimension tables.
- **Fact_Deliveries** is connected to **Dim_Venue** to analyze spatial data (City, Country).
- **Fact_Deliveries** is connected to **Dim_Match** to analyze match-level summaries and season contexts.
- **Fact_Deliveries** is connected to **Dim_Player** to analyze individual batsman and bowler performances.
- **Fact_Deliveries** is connected to **Dim_Date** to allow robust time-series analysis (e.g., performance across different years or months).
- **Fact_Deliveries** is connected to **Dim_Team** twice (via **Batting_Team_SK** and **Bowling_Team_SK**). In this context, **Dim_Team** acts as a **Role-Playing Dimension**, allowing independent analysis of the batting and bowling franchises within the same delivery.
- All dimensional tables utilize auto-incrementing **Surrogate Keys** to maintain historical integrity and isolate the data warehouse from operational source system changes.

Key Elements of the Dimensional Model

Fact Tables	Dimensions
Fact_Deliveries	Dim_Venue, Dim_Match, Dim_Player, Dim_Team, Dim_Date

Slowly Changing Dimension (SCD)

Dim_Venue is implemented as a Slowly Changing Dimension (SCD Type 2) to capture historical changes, such as stadium names changing due to new sponsorships or administrative updates over time.

Assumption / Design Decision:

In the data warehouse design, **Dim_Venue** was chosen as the Slowly Changing Dimension (SCD). This decision was made because venue attributes, specifically stadium names, are subject to change across different IPL seasons. By implementing Dim_Venue as an SCD Type 2 (utilizing Start_Date, End_Date, and Is_Current attributes), it becomes possible to track historical changes in venue data accurately. This ensures that past match records correctly reflect the stadium name as it was at that specific time, rather than overriding it with the current name.

Task 5: ETL Development

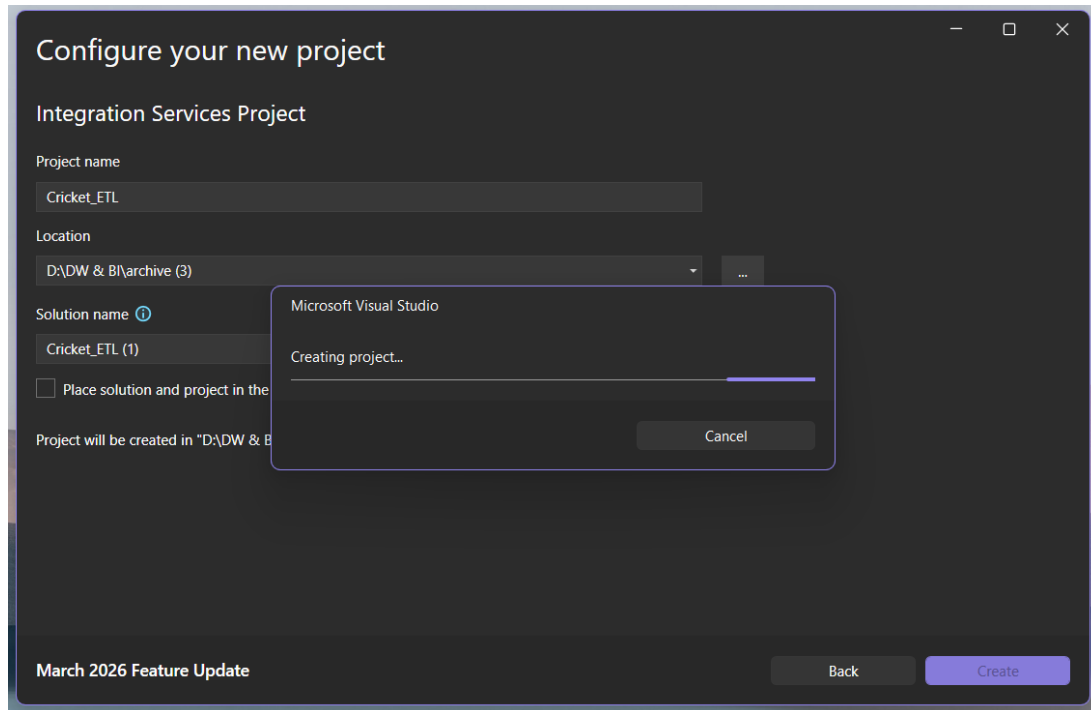
In the ETL (Extract, Transform, Load) process:

1. Extract → Collects raw data from various heterogeneous sources.
2. Transform → Cleans, standardizes, modifies, and corrects the data (this is where data conversion and business logic application happen).
3. Load → Inserts the clean, final data into the dimensional data warehouse.

Extracting

In this project, extraction was performed to collect and organize cricket data from multiple source types, including SQL Server tables, CSV, and Excel files, into the source environment. The data was then loaded into specific staging tables in the Cricket_Staging database, separating match details, deliveries, and venue information. This structured extraction enabled easier cleaning, validation, and transformation before final loading into the data warehouse (Cricket_DW), ensuring data consistency and readiness for dimensional modeling.

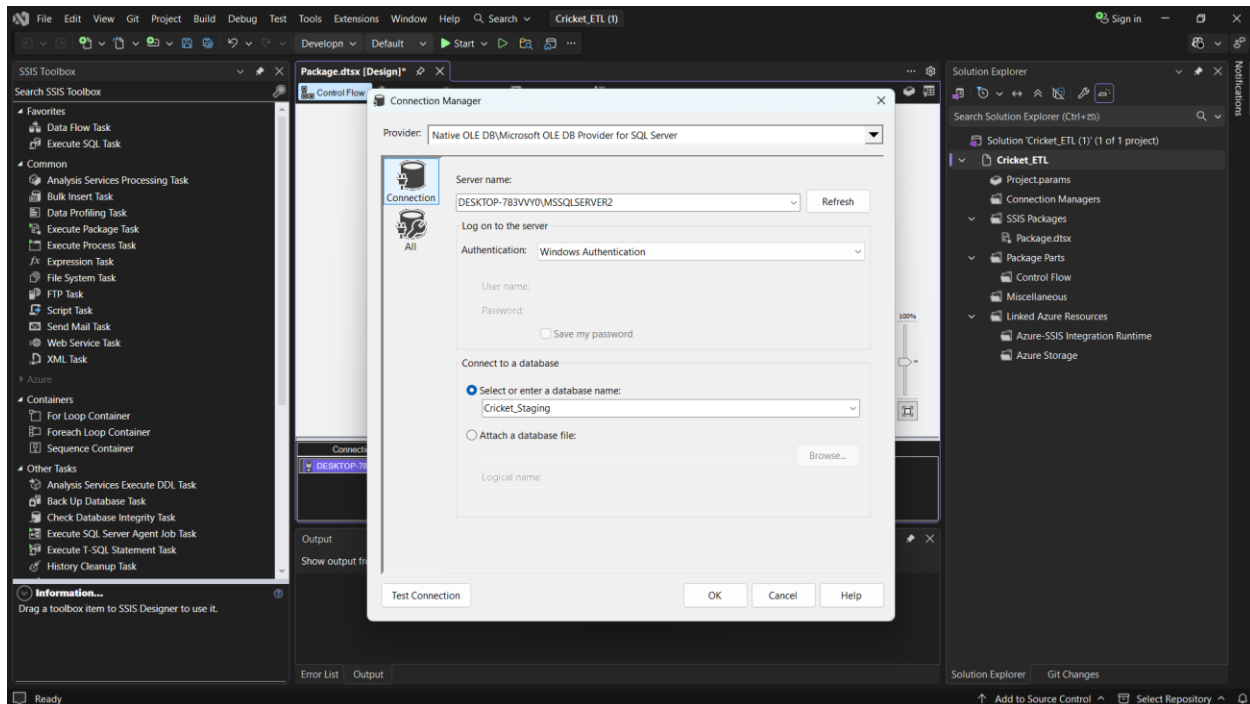
Staging is done during the extracting step to store the raw extracted data in temporary tables before transformation.



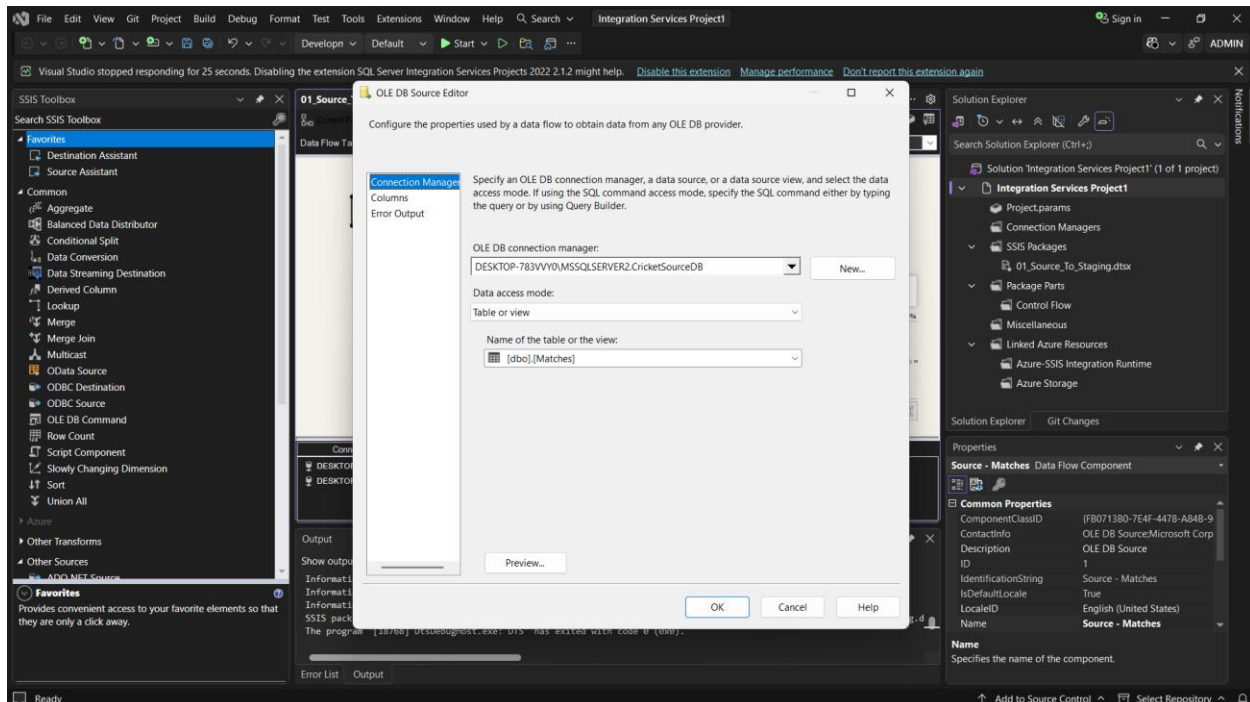
OLE DB Source / Flat File Source / Excel Source: Used to read data from the respective source files or databases (e.g., the Matches table in the source DB or the deliveries.csv file).

OLE DB Destination: Used to write (insert) data into a SQL table. Here, it dynamically created and wrote data into the staging tables (e.g., Stg_Matches) in the Cricket_Staging database.

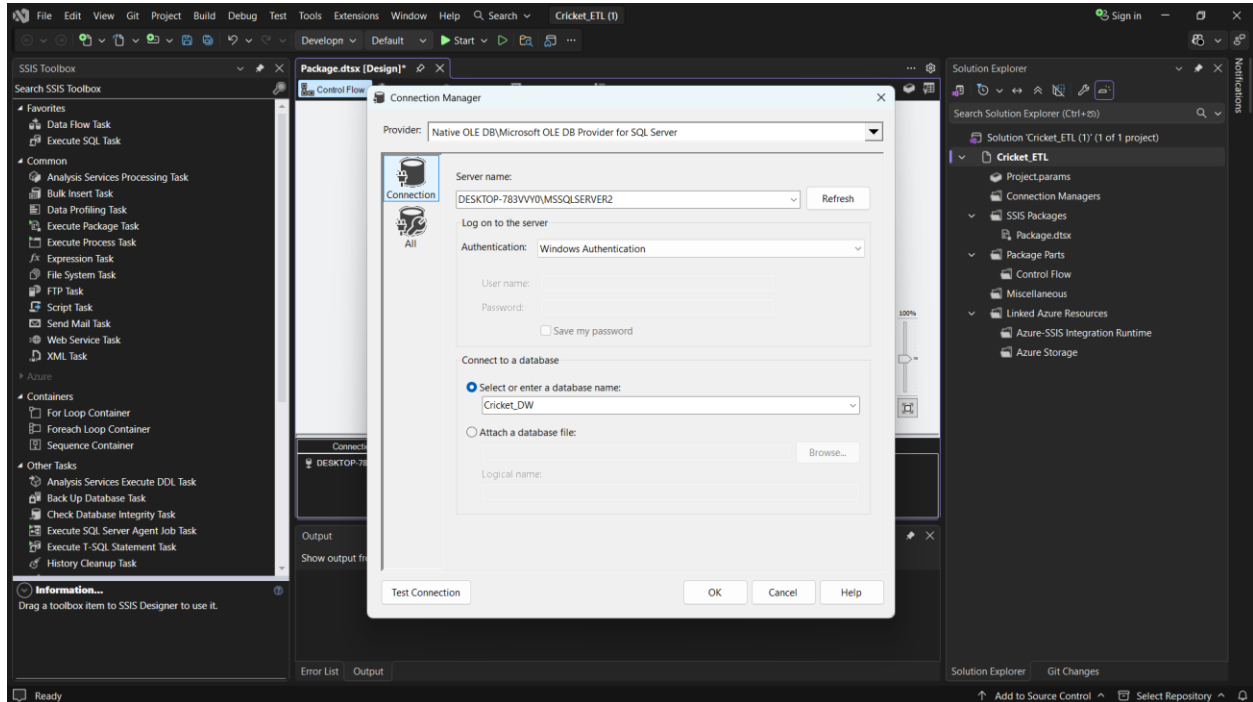
Made the Test Connection in OLE DB Source



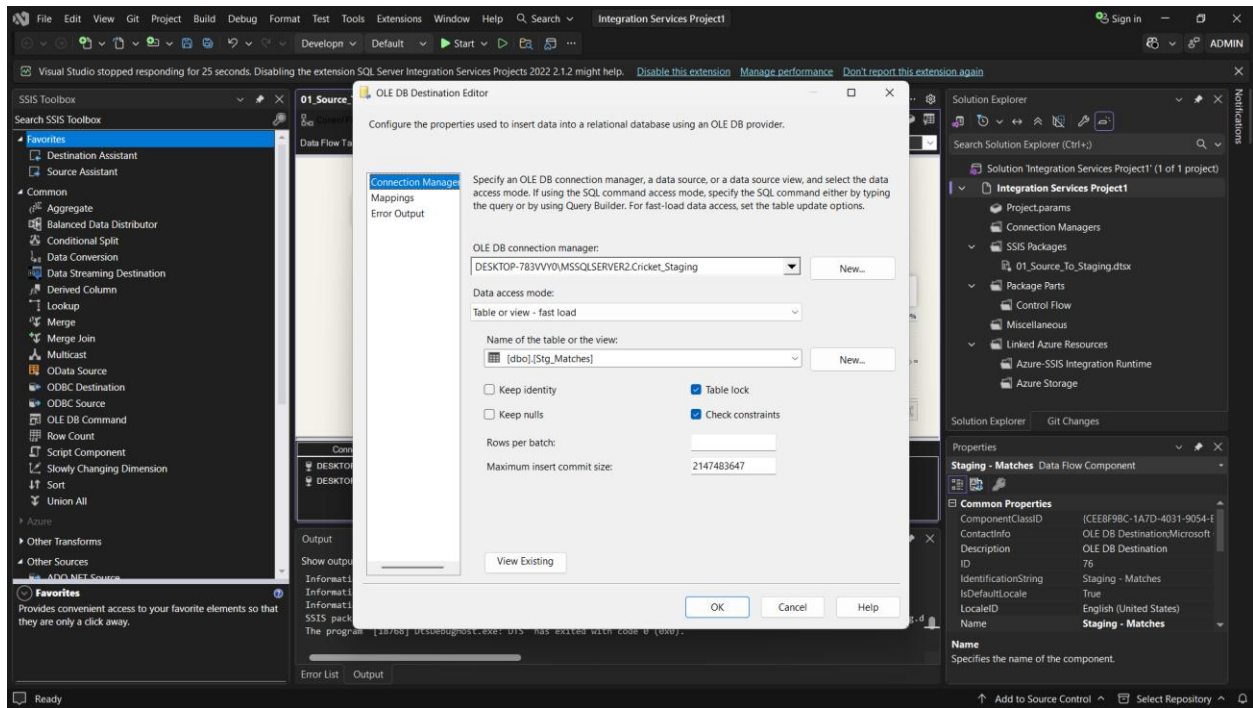
First the Stg_Matches Staging Table



Making the Connection for OLE DB Destination

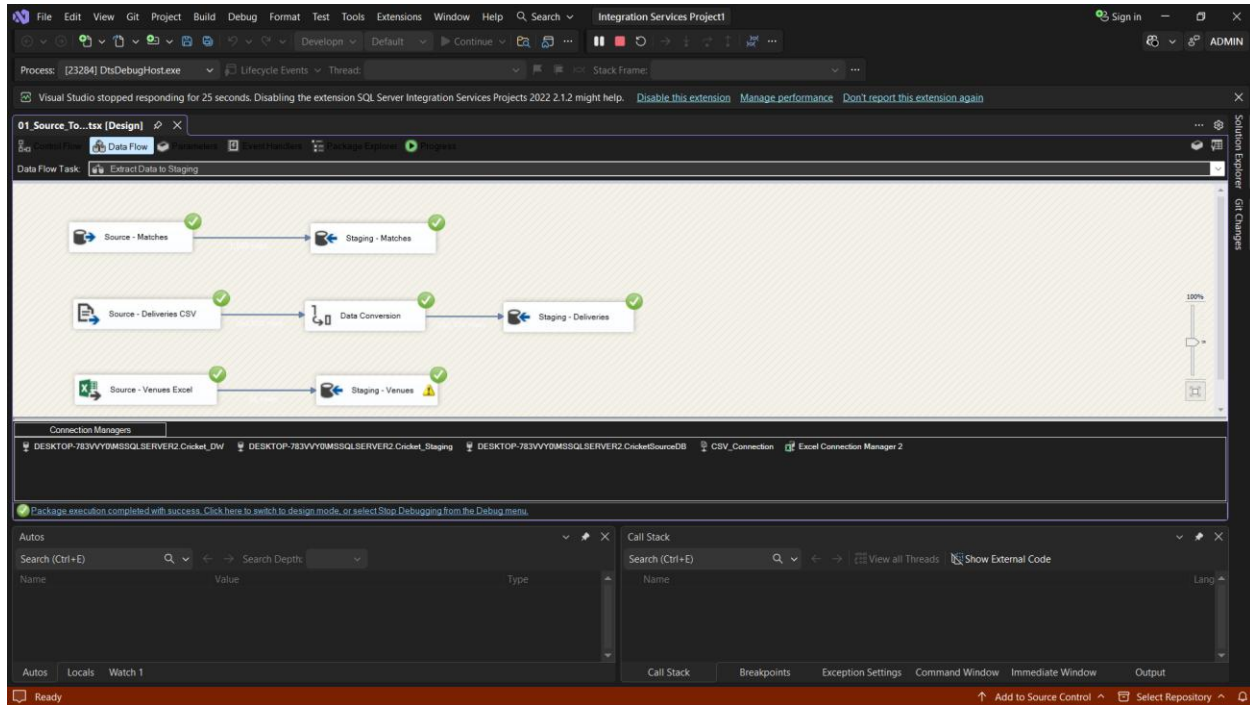


Selected Stg_Matches Staging for OLE DB Destination



The same steps were repeated for every staging table and for each a data flow was created as depicted below.

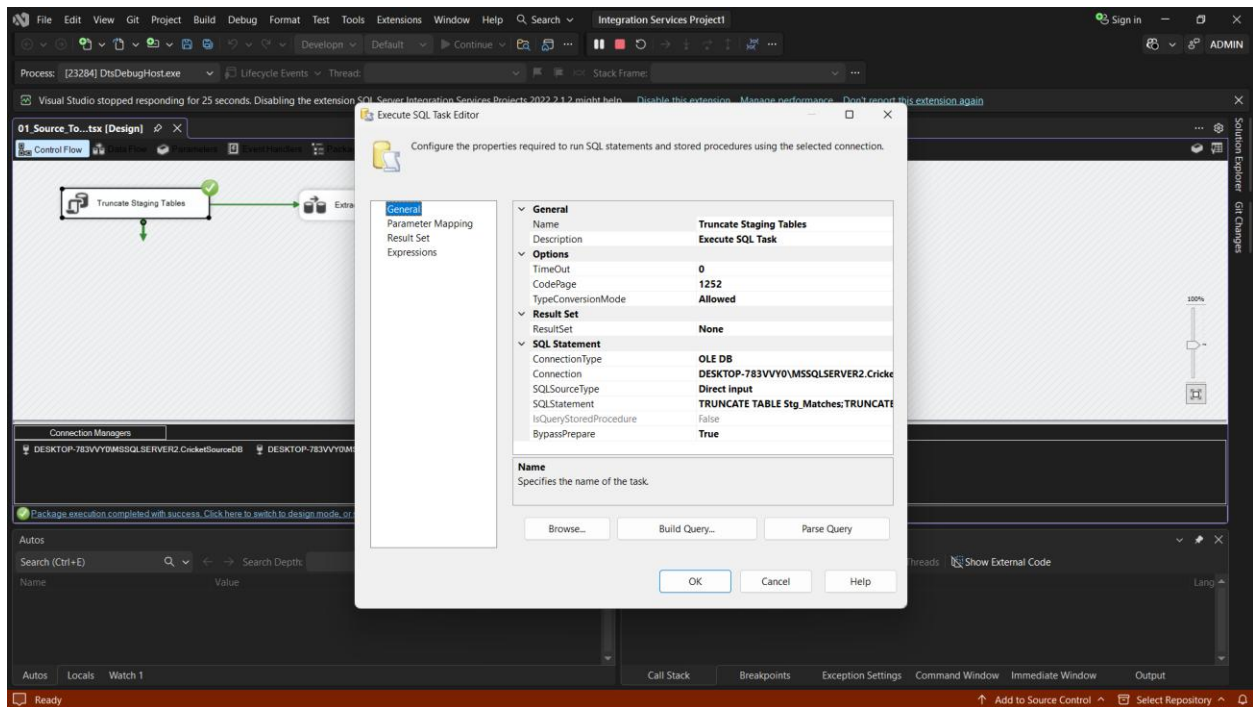
Final Output after Extracting,



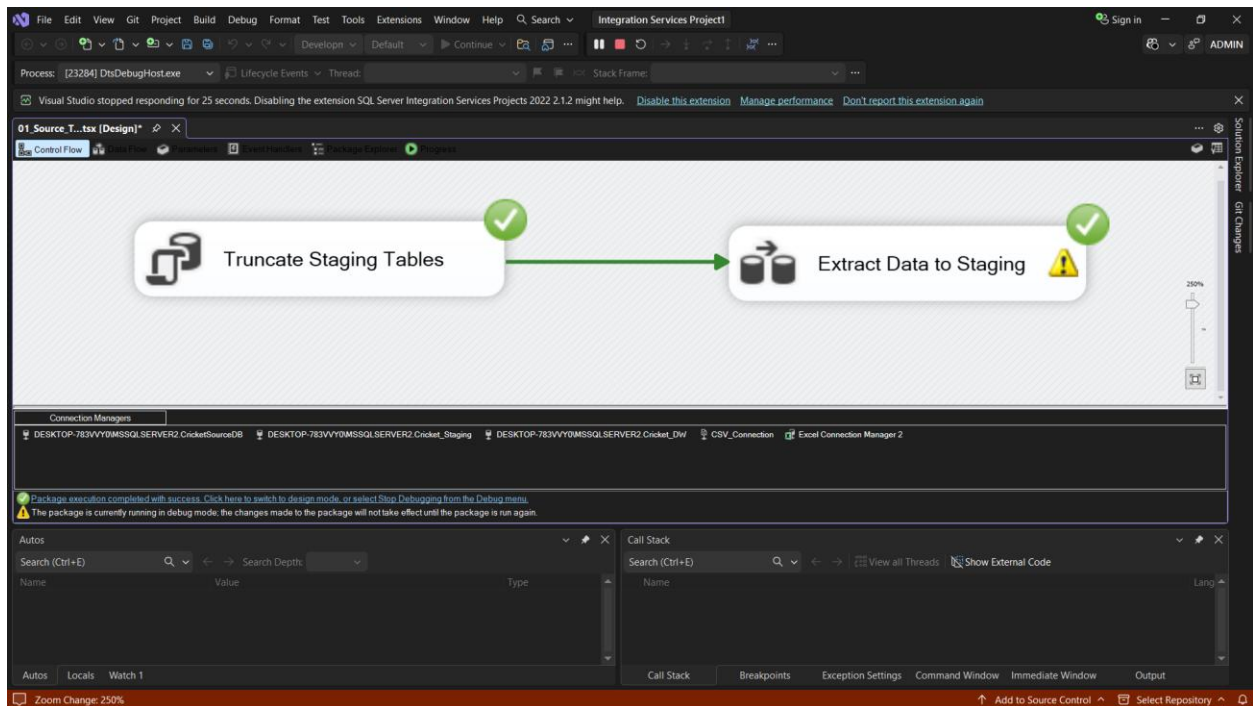
Truncate Table

In this project, the TRUNCATE TABLE command was used during the ETL process to clear the staging tables in the Cricket_Staging database before reloading fresh data. This ensures that every time the ETL package runs, the staging tables start empty, preventing old or duplicate data from remaining in the staging area.

By truncating staging tables before each data load, the data extraction and transformation processes could proceed cleanly, guaranteeing data accuracy and avoiding duplication in the final Data Warehouse. This was achieved using the Execute SQL Task component in the SSIS Control Flow.

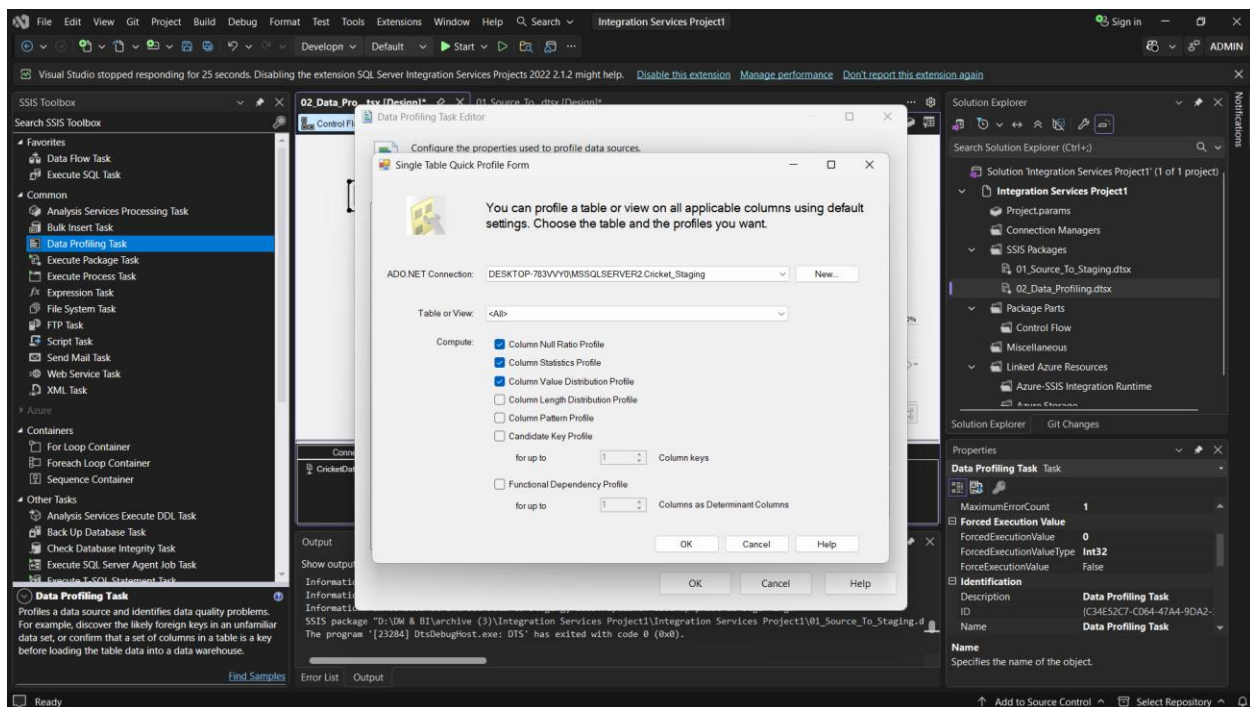


Final Output after Truncating the Table,

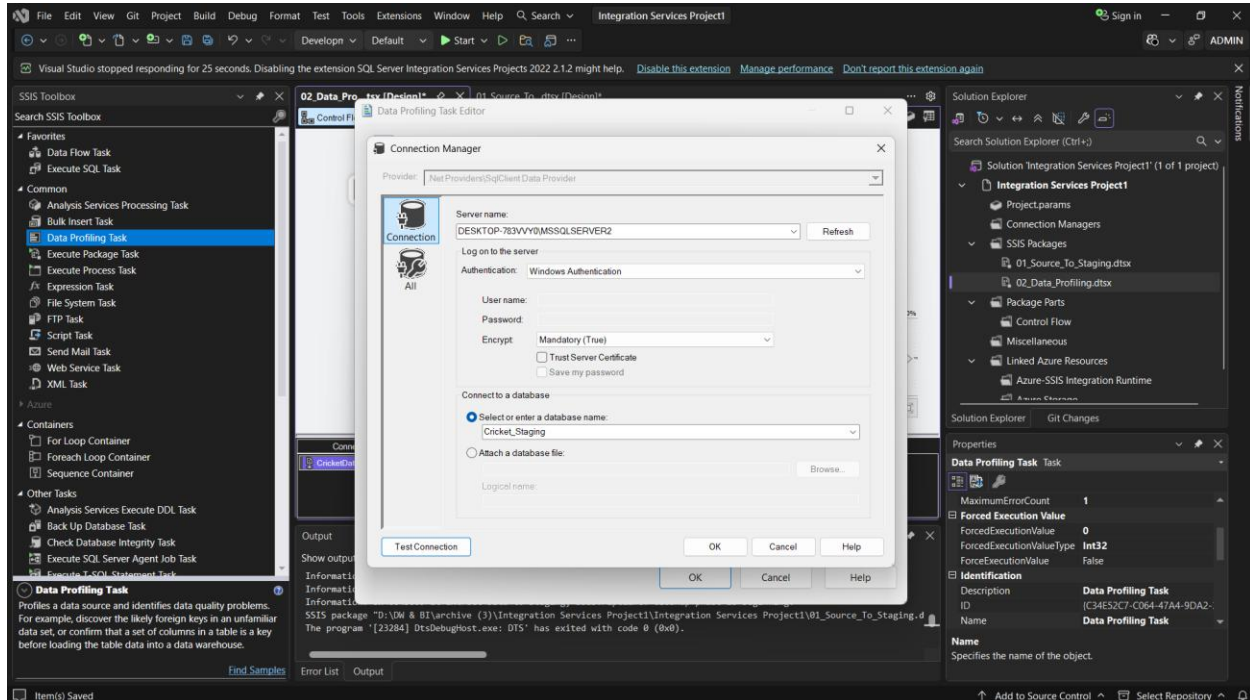


Data Profiling

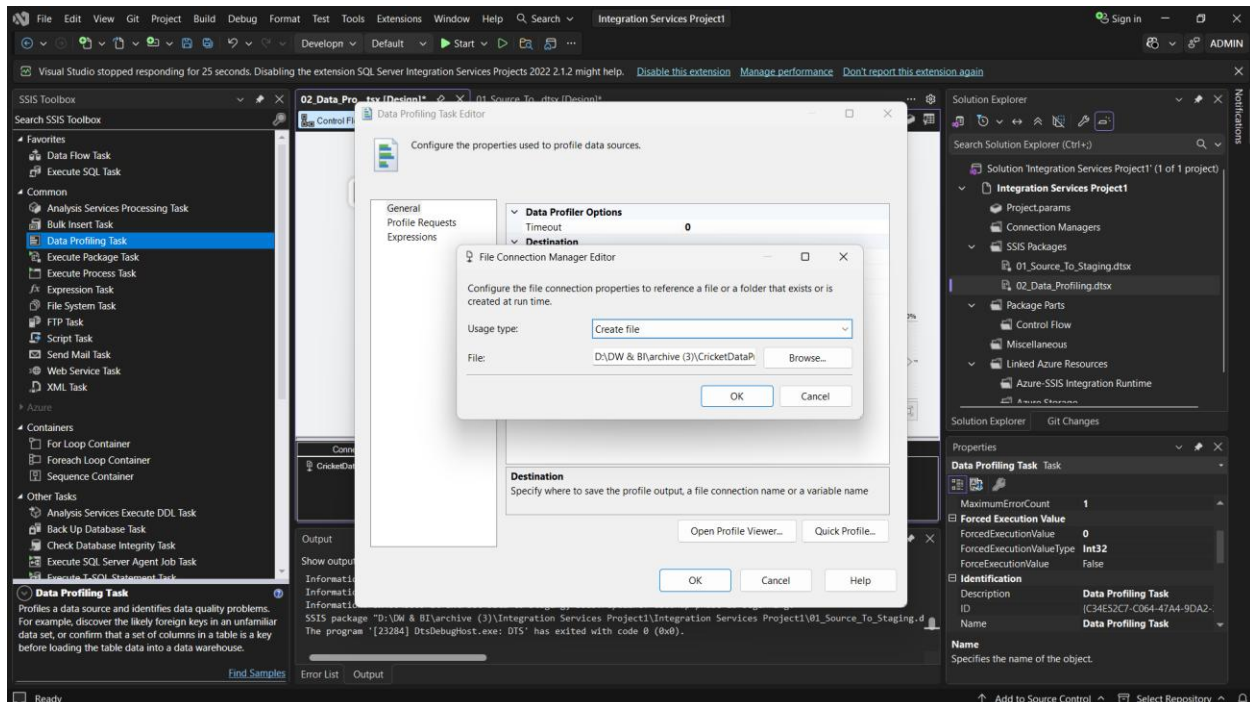
Data profiling was performed on the staging tables after the extraction process to better understand the quality, completeness, and structure of the data loaded from the raw CSV and Excel files. By profiling the data in the Cricket_Staging database, potential issues such as NULL values, datatype mismatches, inconsistent naming conventions, and data anomalies were identified early. This step was crucial to ensure that fields like match_id, batter, bowler, venue, and total_runs were accurate and aligned with the dimensional schema design. Data profiling also verified that surrogate keys and foreign key relationships could be established during the lookup transformations without data integrity errors. Based on these profiling insights, necessary data validation steps were identified before the final load into the Cricket_DW data warehouse, ensuring high-quality, reliable data for analytical reporting.



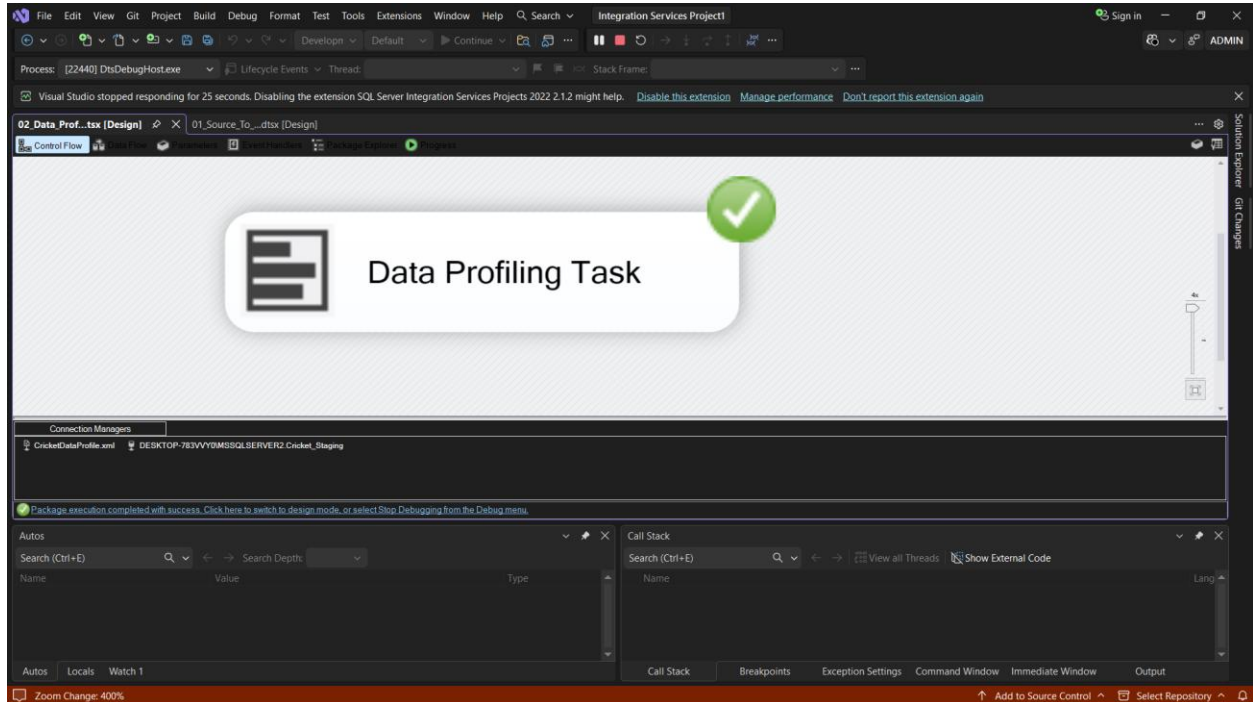
Made a New Connection to Staging



Adding the File



Final Output of Data Profiling



Data Profile Viewer -

Open Refresh

Profiles (Table View)

Data Sources

- DESKTOP-783VVY0MSSQLSERVER2
 - Databases
 - Cricket_Staging
 - Tables
 - [dbo].[Stg_Deliveries]
 - Column Null Ratio Profiles
 - Column Value Distribution Profiles
 - [dbo].[Stg_Matches]
 - Column Null Ratio Profiles
 - Column Value Distribution Profiles
 - [dbo].[Stg_Venues]
 - Column Null Ratio Profiles
 - Column Value Distribution Profiles

Column Null Ratio Profiles - [dbo].[Stg_Deliveries]

Column	Null Count	Null Percentage
ball	0	0.0000 %
batsman_runs	0	0.0000 %
batter	0	0.0000 %
battering_team	0	0.0000 %
bowler	0	0.0000 %
bowling_team	0	0.0000 %
dismissal_kind	0	0.0000 %
extra_runs	0	0.0000 %
fielder	0	0.0000 %
inning	0	0.0000 %
is_wicket	0	0.0000 %
match_id	0	0.0000 %
non_striker	0	0.0000 %
over	0	0.0000 %
player_dismissed	0	0.0000 %
total_runs	0	0.0000 %

Successfully loaded data profile from ...

Message

Transform and Load Data

After extracting and profiling the data in the staging area (**Cricket_Staging**), various transformations were applied to clean and standardize the data. Data conversion tasks were performed to fix datatype mismatches, such as explicit type casting to convert string date fields into proper date formats for time-based analysis. Other transformations included handling null values, removing duplicate records using distinct queries, and mapping fields to match the data warehouse schema. Once the data was properly transformed, it was loaded into the final dimensional data warehouse (**Cricket_DW**). During the loading process, surrogate keys were generated automatically for dimension tables via identity columns, and the central fact table, **Fact_Deliveries**, was populated by correctly establishing relationships with dimension tables such as **Dim_Match**, **Dim_Venue**, **Dim_Player**, **Dim_Team**, and **Dim_Date** using Lookup transformations. This ensured the data was clean, consistent, and optimized for OLAP cube building and reporting.

Ø Transform and Loading Dim_Date

Transform data from your staging table (Stg_Matches) and load into the dimension table (Dim_Date) using:

- OLE DB Source
- Derived Column
- OLE DB Destination

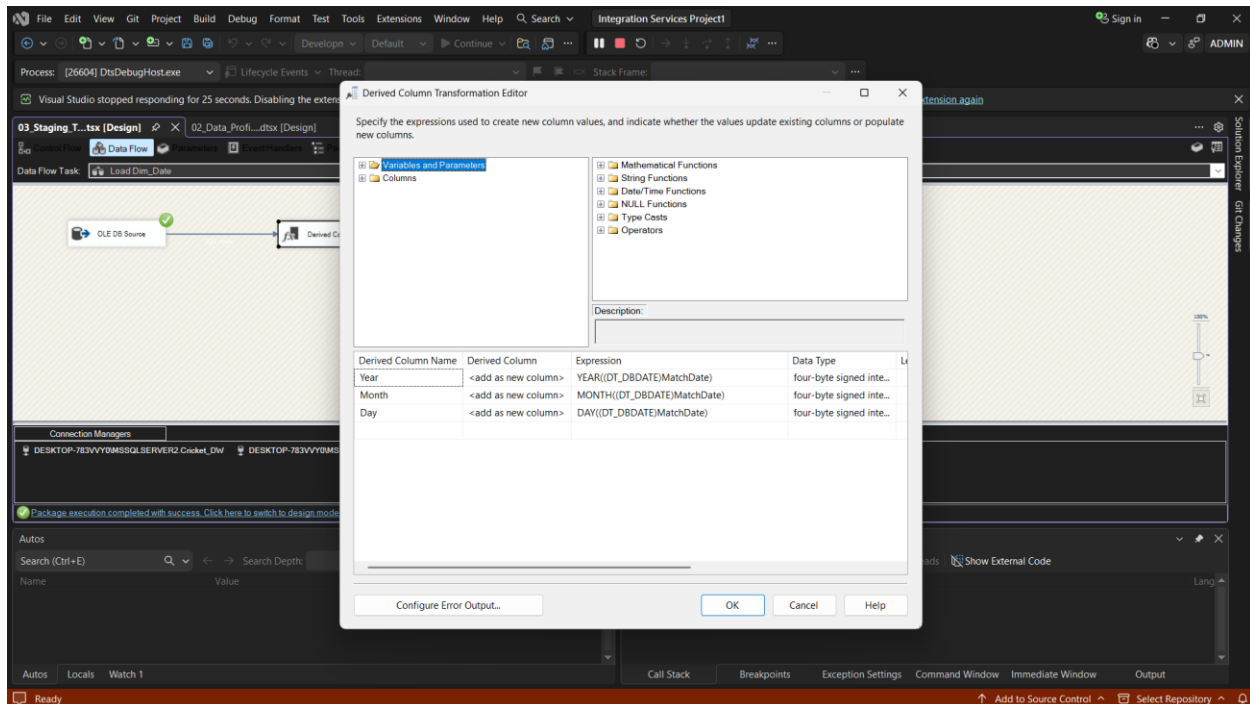
OLE DB Source is a component used in SSIS (SQL Server Integration Services) to read (fetch) data from a database. In this ETL process, the OLE DB Source was used to extract data from staging tables (in your **Cricket_Staging** database). It connects to SQL Server using an OLE DB connection manager and pulls distinct rows that you want to transform and load into your final warehouse. Example: Reading the raw 'date' field from the **Stg_Matches** table.

Derived Column is an SSIS transformation component used to create new columns or replace existing ones by applying expressions to the data flow. For the **Dim_Date** table, it was utilized to extract granular time attributes from a single date string. It was also used to handle datatype mismatches by explicitly casting the string date into a proper database date format. Example: Using expressions like `YEAR((DT_DBDATE)MatchDate)` and `MONTH((DT_DBDATE)MatchDate)` to dynamically generate Year, Month, and Day columns during the pipeline.

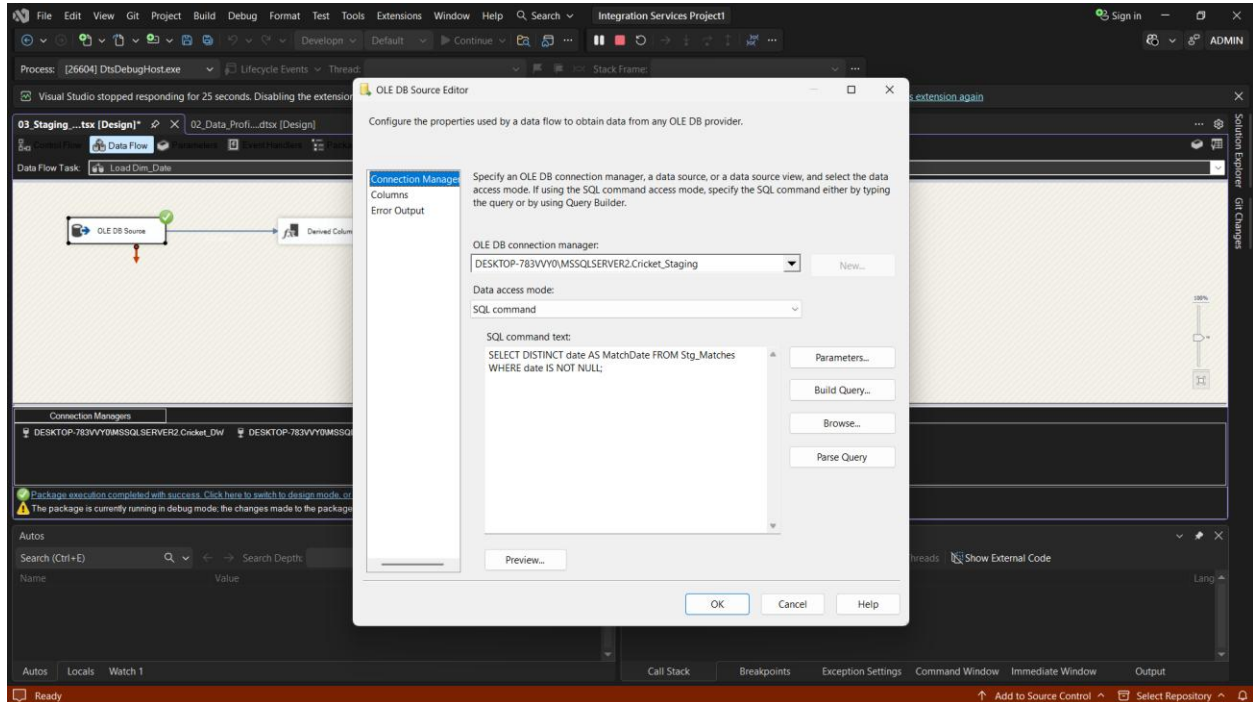
OLE DB Destination is an SSIS component used to bulk-insert data into a destination database. Once the date attributes were successfully extracted and transformed, the OLE

DB Destination was used to fast-load the cleaned records directly into the final dimension table (**Dim_Date**) inside the **Cricket_DW**. This prepares the time-based hierarchical data required for OLAP cube analysis.

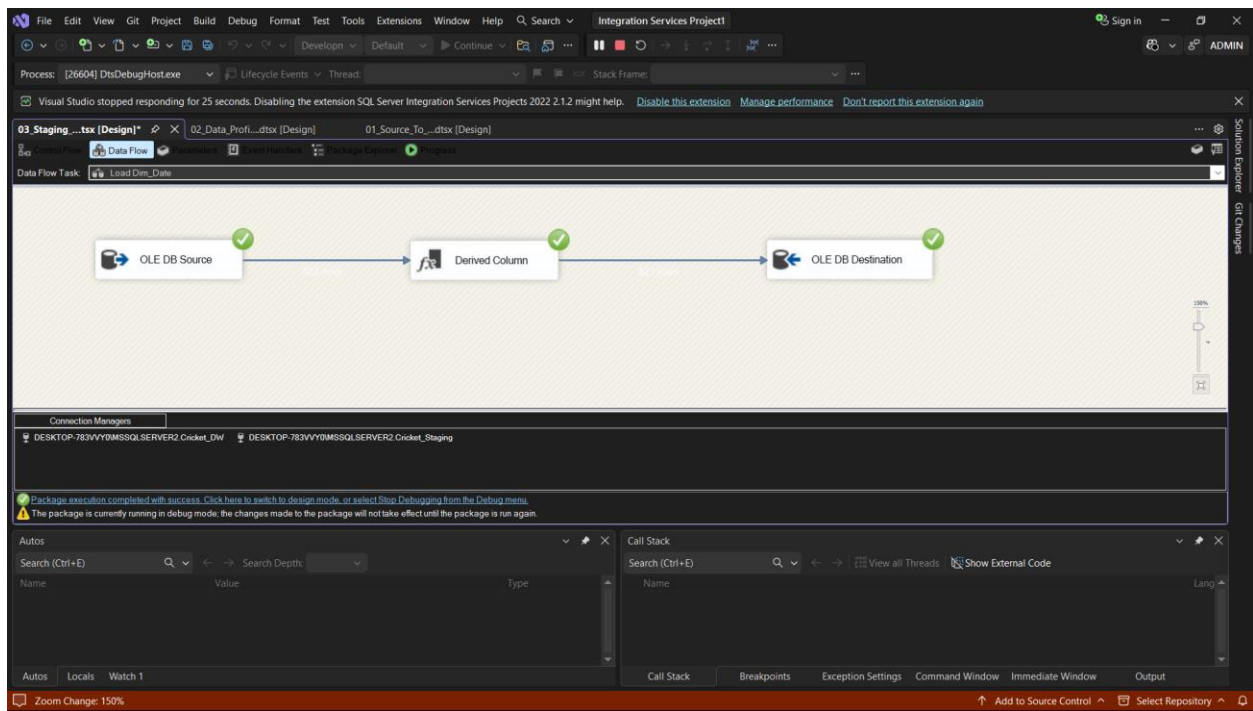
A Derived Column was Made



Selected the Source of the Staging Dim_Date and Selected the Respective Table

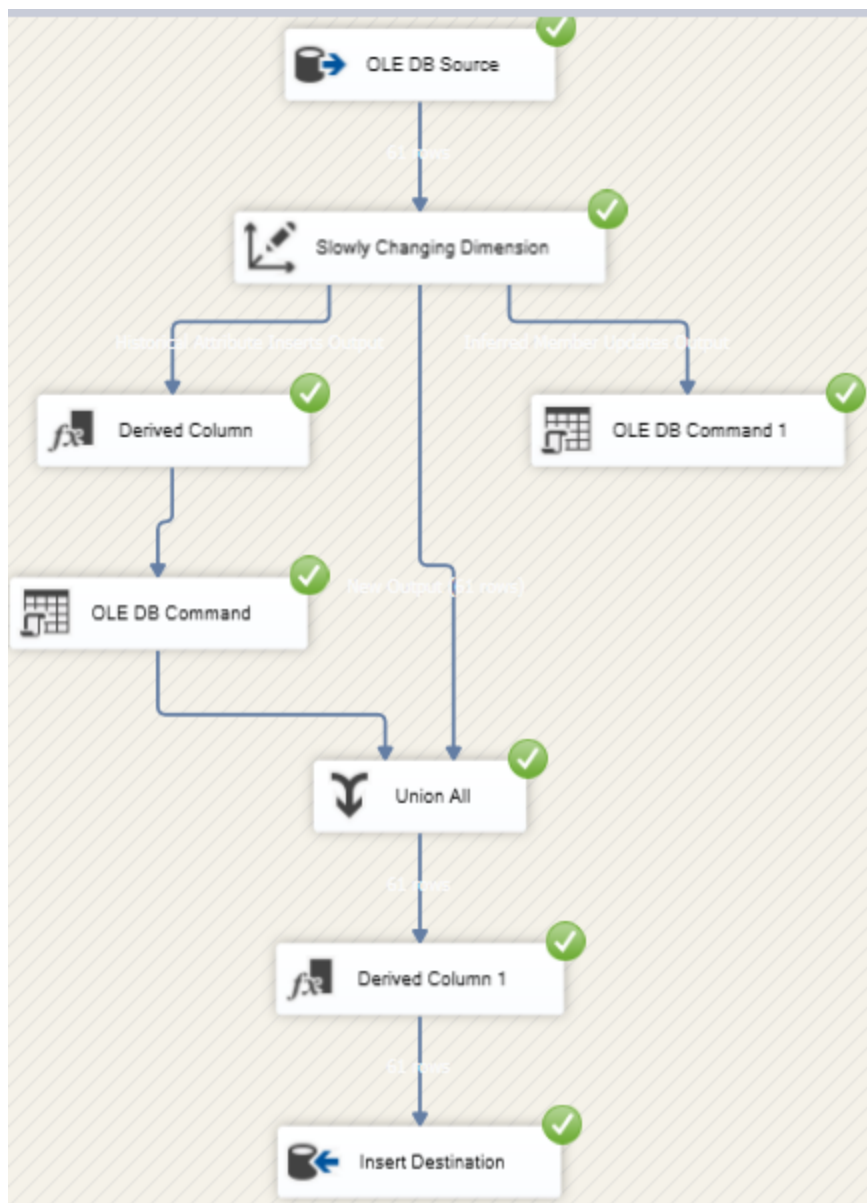


Final Data Flow for Dim_Date (Loaded Dim_Date Successfully)

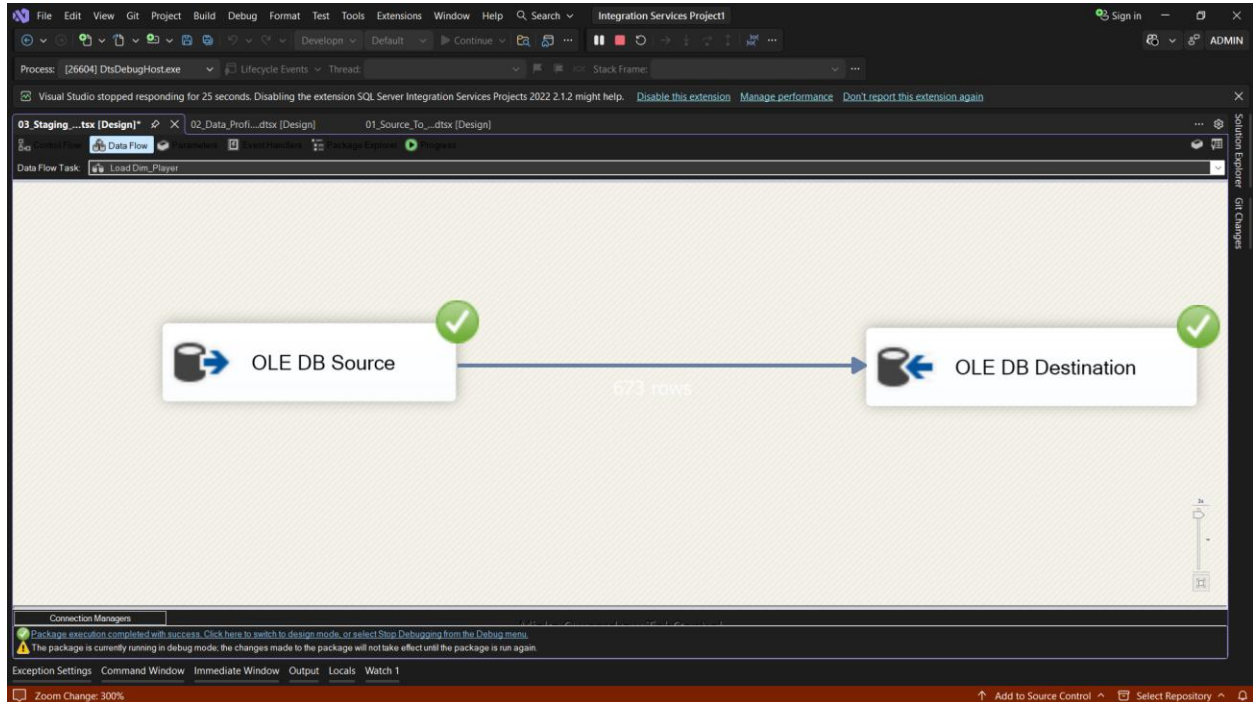


Ø Transform and Loading Dim_Venue (Slowly Changing Dimension)

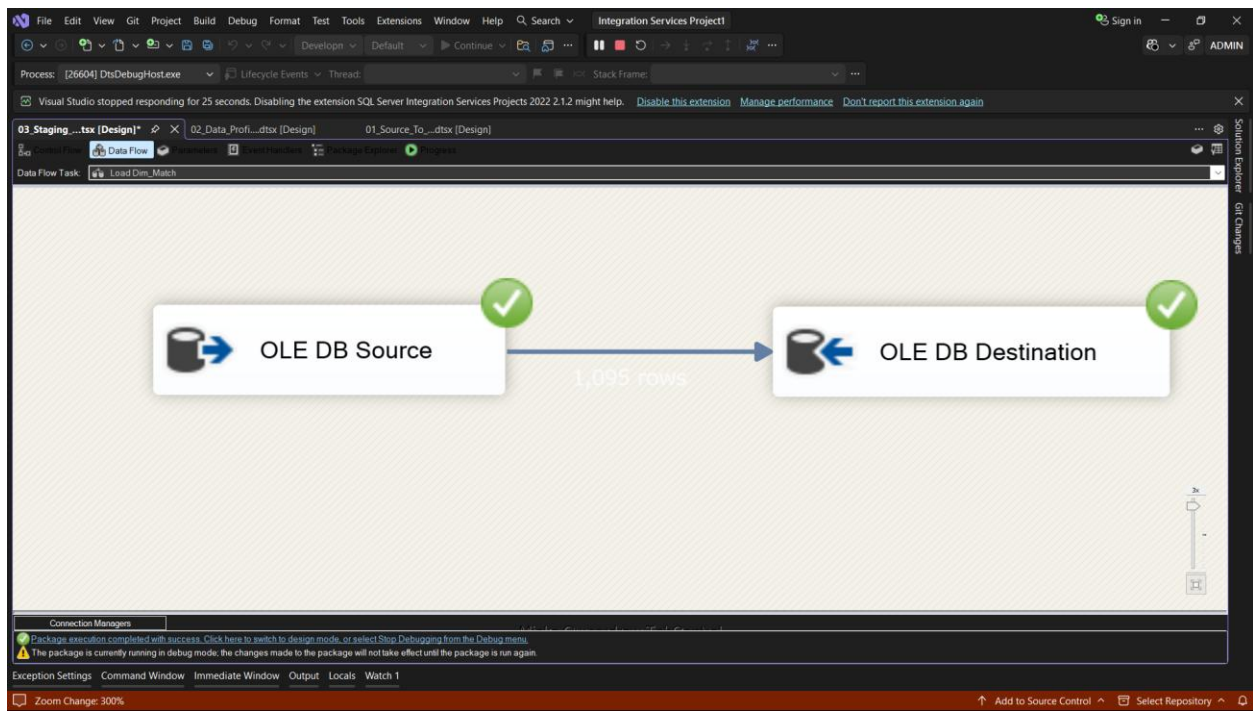
In this project, a Slowly Changing Dimension (SCD Type 2) was used to manage changes in venue data. Attributes such as Venue_Name can change over the years due to sponsorships or administrative updates, and it was necessary to keep track of these historical changes for accurate reporting. Therefore, an SCD transformation was applied to the Dim_Venue table. The Venue_ID (Natural Key) was used as the Business Key, which serves as a unique identifier from the source system. This business key is critical because it is used to match incoming data to existing dimension rows, helping to detect whether a current record should simply be updated or if a new historical version should be inserted to properly maintain the timeline.



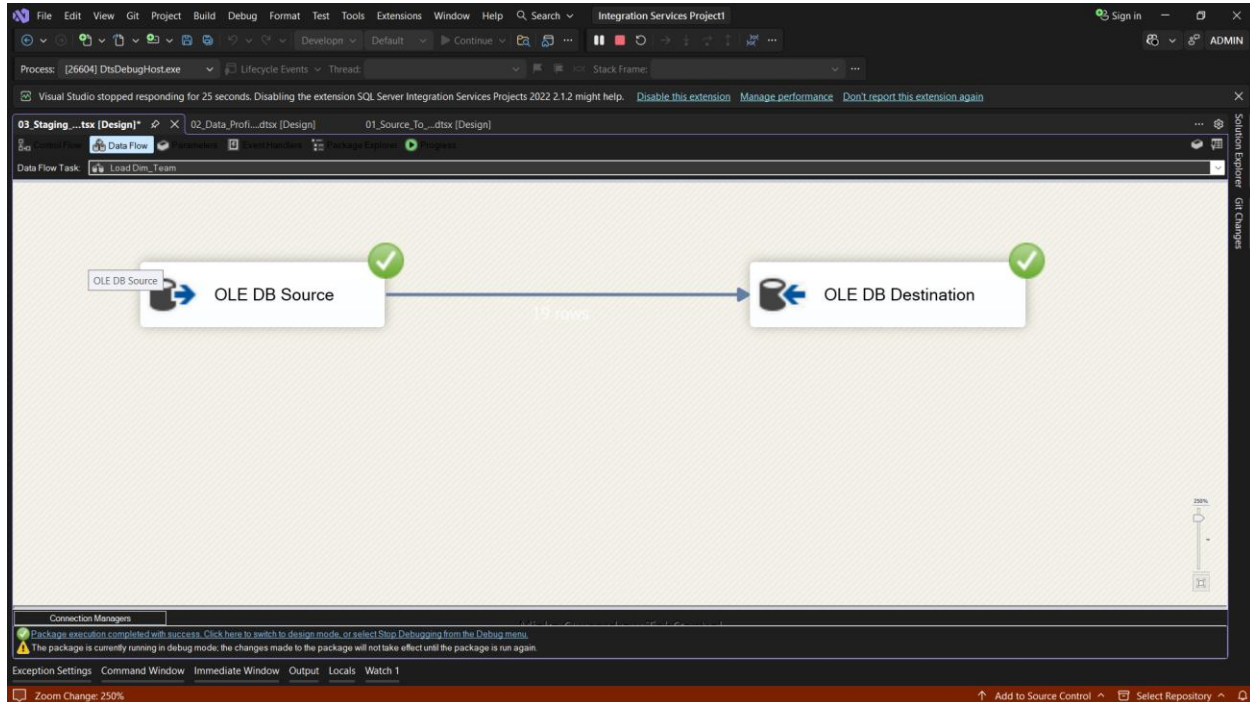
Transform and Loading Dim_Player



Transform and Loading Dim_Match



Transform and Loading Dim_Team



Transform and Loading Fact_Deliveries

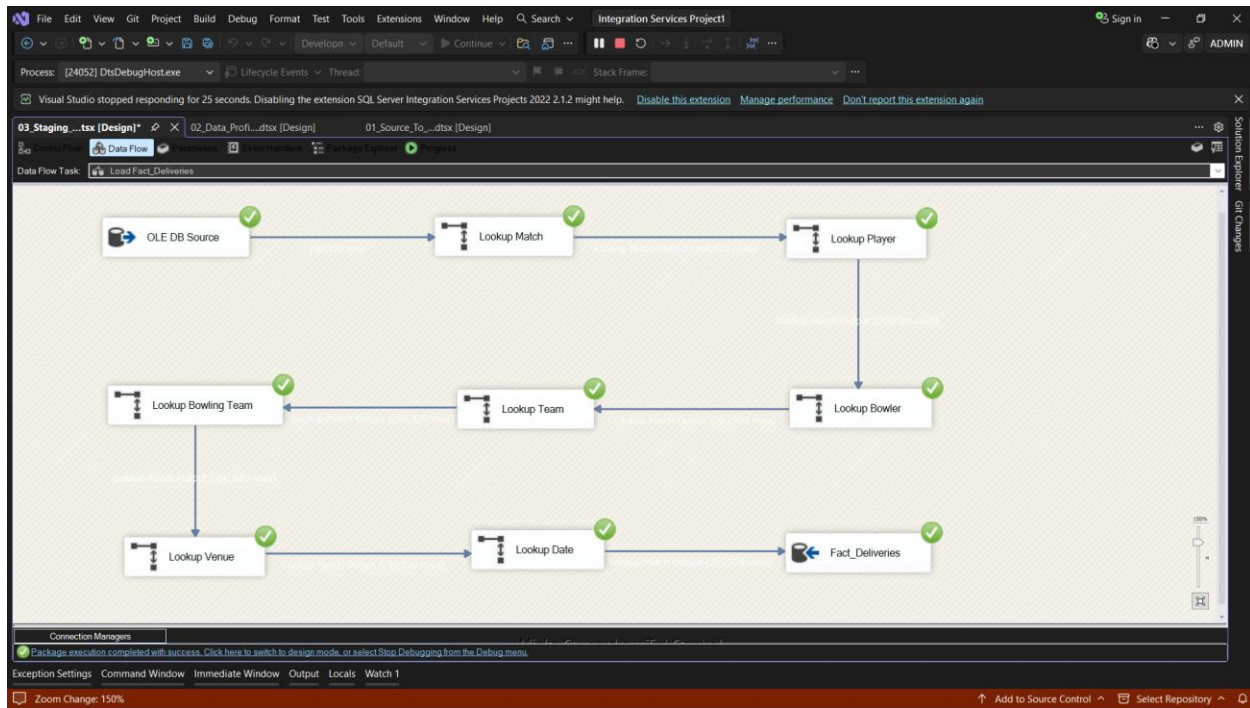
In the ETL flow for **Fact_Deliveries**, a total of **7 Lookup transformations** were implemented. This was necessary to ensure every granular delivery record is correctly mapped to its respective surrogate keys across all dimensions in the Star Schema.

Specifically, the lookups were used for:

1. **Dim_Match Lookup:** To retrieve Match_SK using the Match_ID.
2. **Dim_Venue Lookup:** To retrieve Venue_SK using the Venue_ID.
3. **Dim_Player (Batsman) Lookup:** To retrieve the surrogate key for the player who was batting.
4. **Dim_Player (Bowler) Lookup:** To retrieve the surrogate key for the player who was bowling.
5. **Dim_Team (Batting Team) Lookup:** To retrieve the surrogate key for the franchise currently batting.

6. **Dim_Team (Bowling Team) Lookup:** To retrieve the surrogate key for the franchise currently bowling.
7. **Dim_Date Lookup:** To map the delivery to the correct record in the date dimension using the match date.

By implementing these 7 lookups, I transformed the raw transactional data into a fully relational format within the data warehouse. This multi-lookup approach allows for deep analytical slicing, such as analyzing a specific bowler's performance against a specific batting team at a particular venue.



Error Handling in ETL

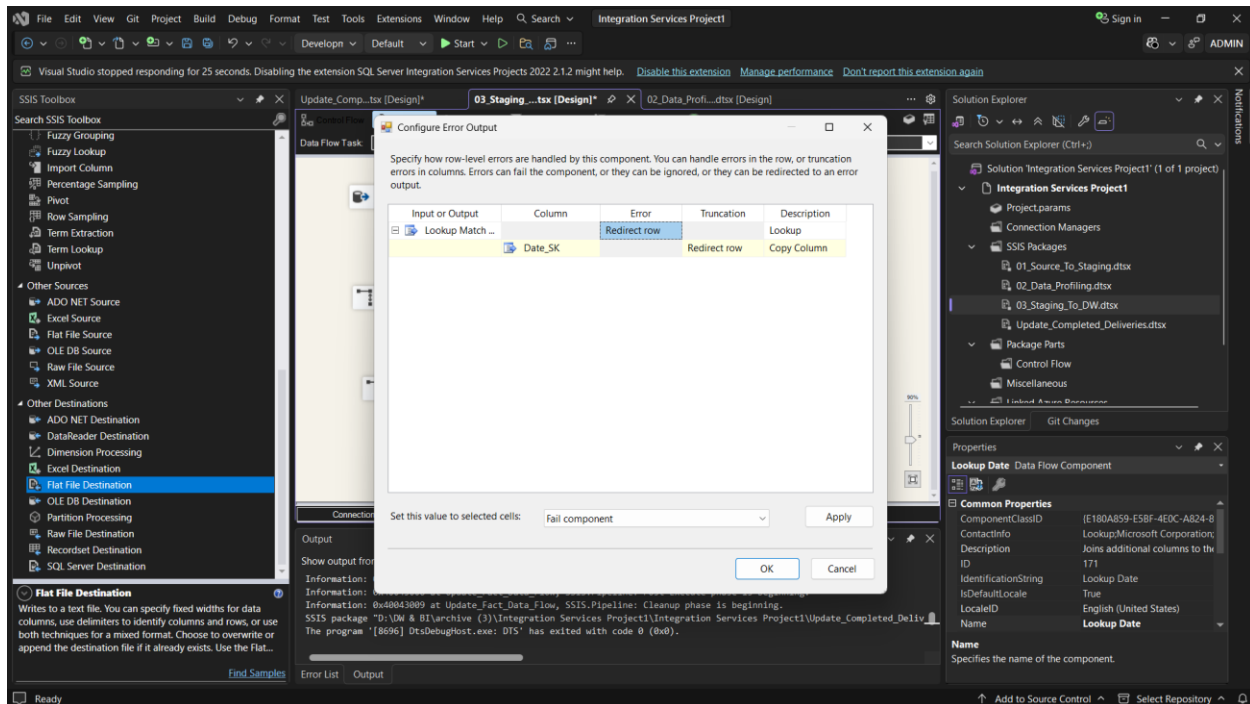
Robust error handling was implemented within the SSIS packages to ensure that data quality issues or transformation failures do not cause the entire ETL pipeline to crash. In a real-world scenario like sports analytics, raw data anomalies (such as an unmatched player name or an invalid date format) are common, and managing them gracefully is a critical part of data warehouse design.

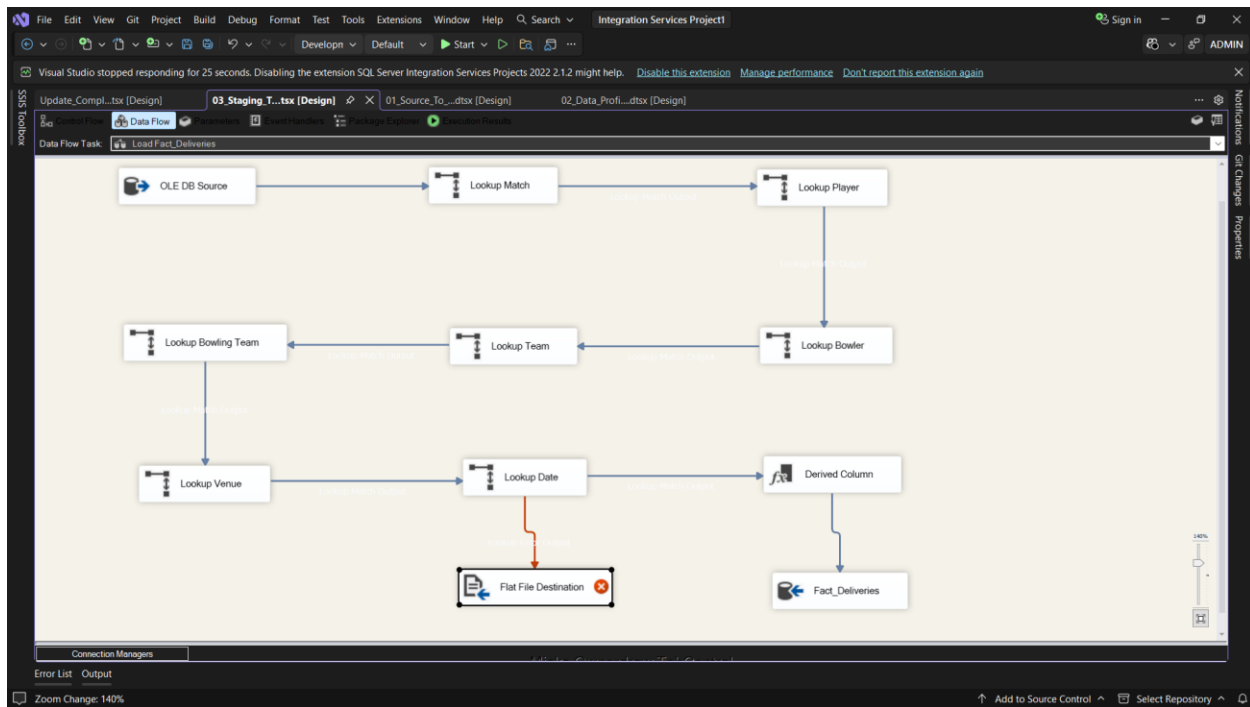
During the transformation and loading process (particularly within **Lookup Transformations** and **Data Conversions**), the "**Configure Error Output**" feature was utilized. Instead of using the default "Fail Component" behavior, the error rows were managed systematically.

Error Handling Strategy Applied:

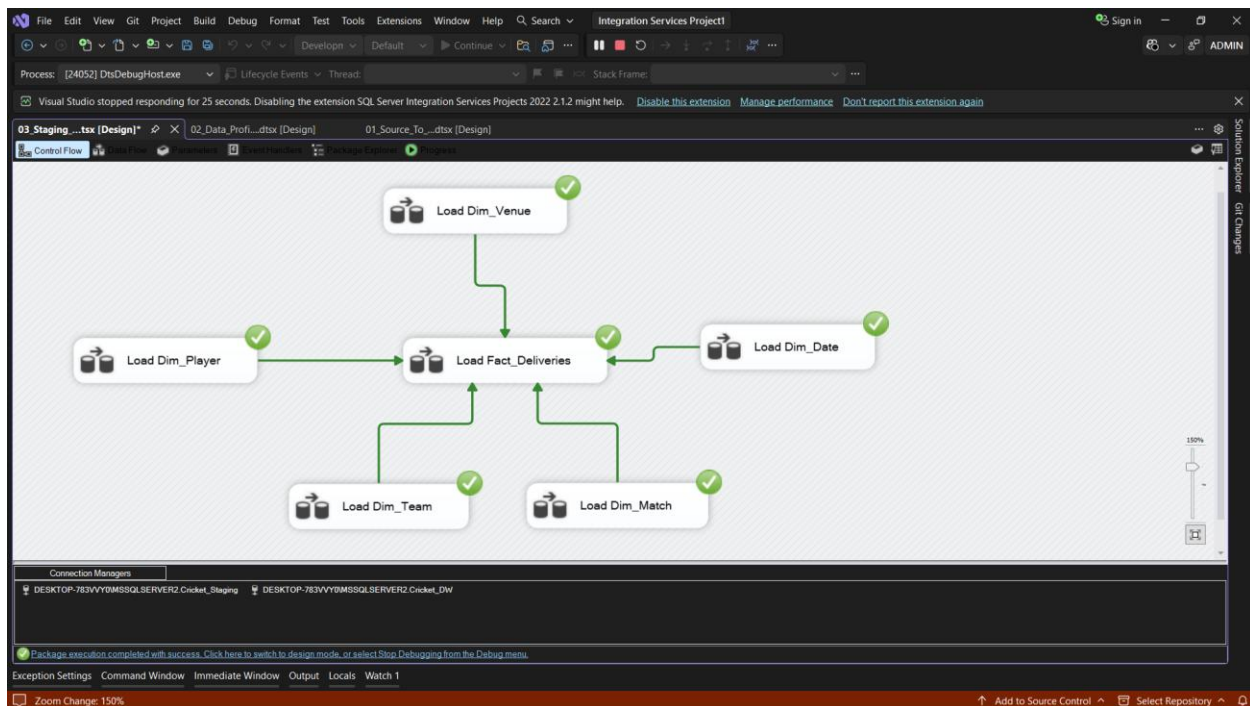
- **Redirect Row:** For critical components like the multiple lookups in the Fact_Deliveries data flow, the error output behavior was set to **"Redirect row"**.
- If an incoming record failed to match a surrogate key (e.g., a completely new player name not present in the Dim_Player table), that specific row was diverted through the red error output path.
- These unresolved rows can be redirected into a separate "Error Log" flat file or a staging error table for further administrative review and manual correction, ensuring that the main pipeline continues to load the valid data without interruption.

This approach guarantees high data integrity in the Cricket_DW while providing a clear audit trail of data anomalies.



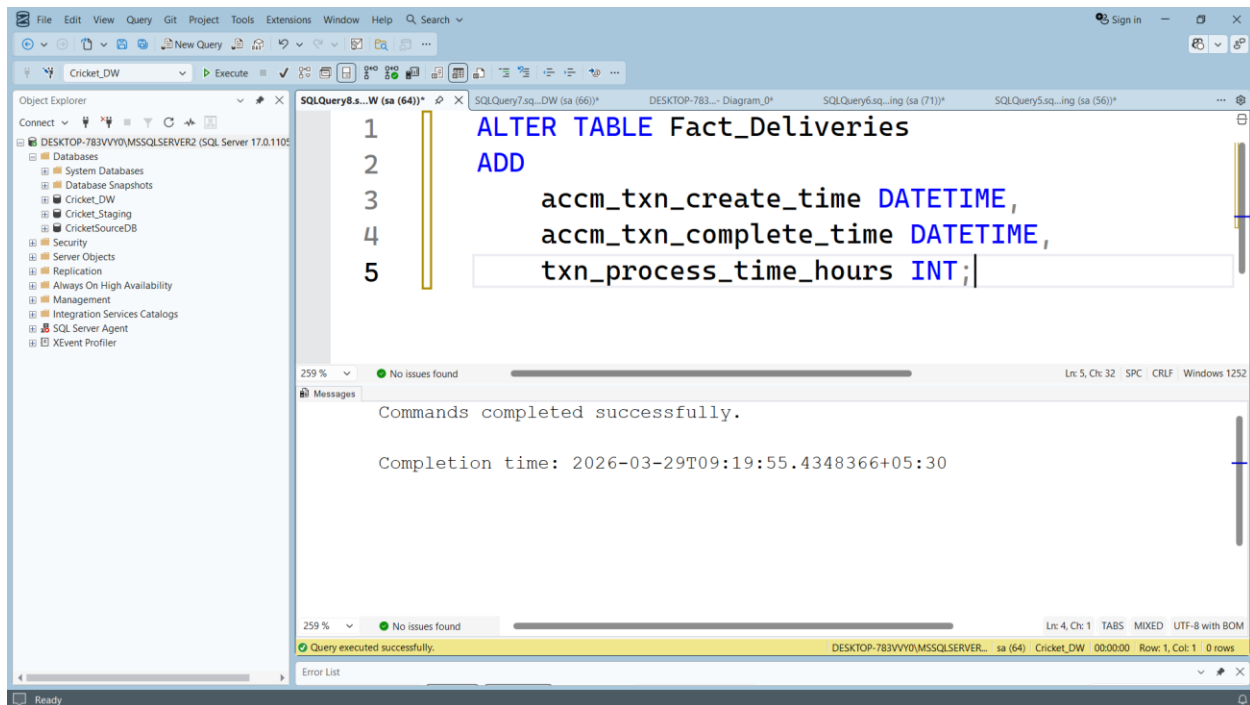


Final Output Transformed and Loaded to Data Warehouse



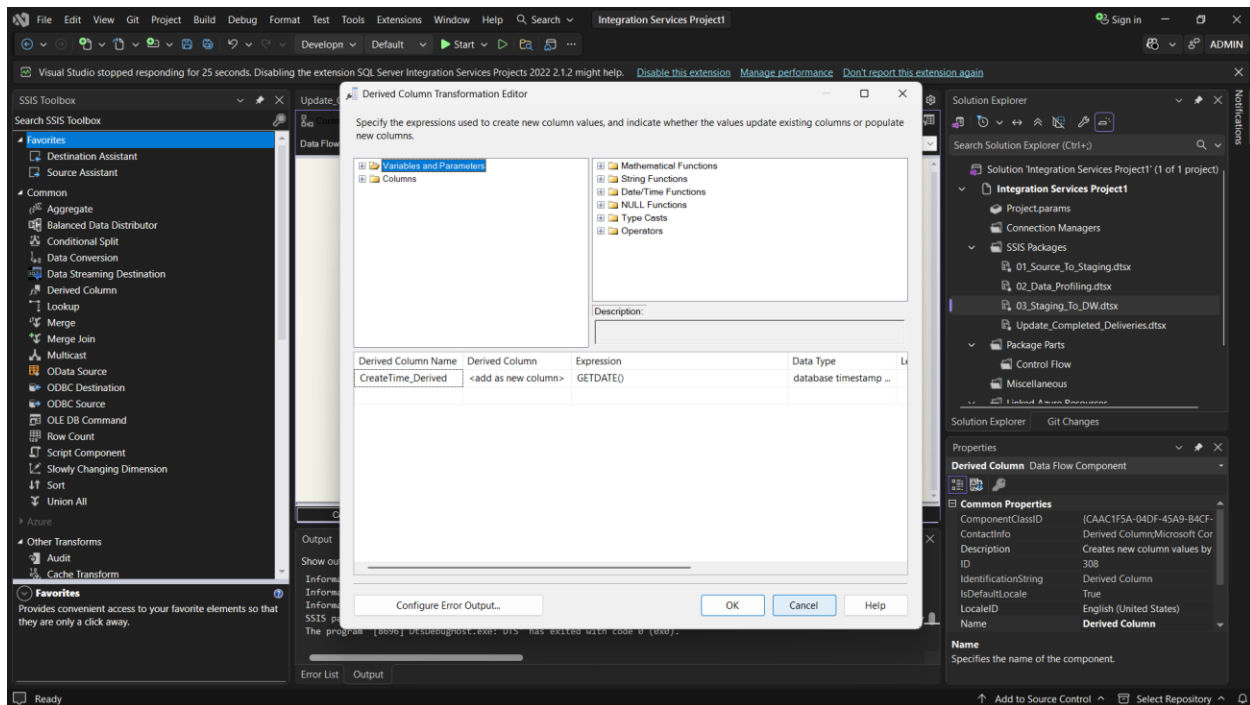
Task 6: ETL Development – Accumulating Fact Tables

ALTER the Fact Table (Extend Columns)

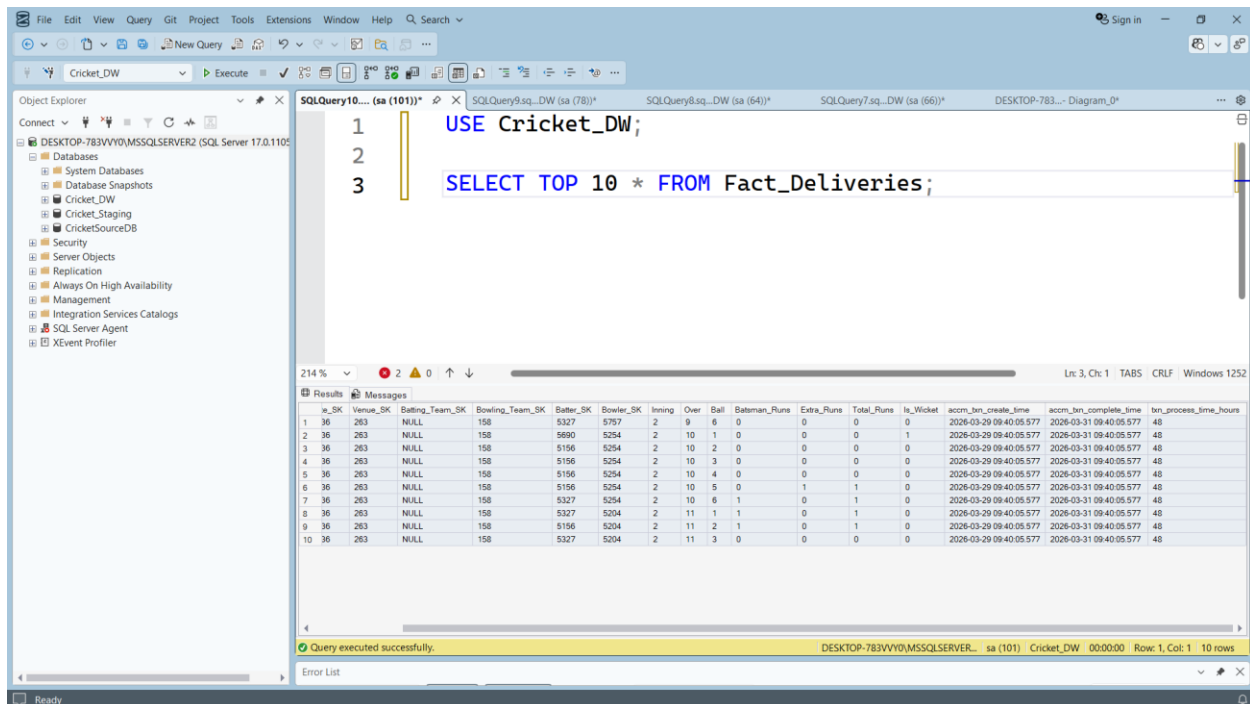


Set accm_txn_create_time During Initial Load

The `accm_txn_create_time` indicates the actual time when the record was first created in the system. During the initial ETL load, I used a **Derived Column** transformation with the `GETDATE()` function to capture the current system timestamp.

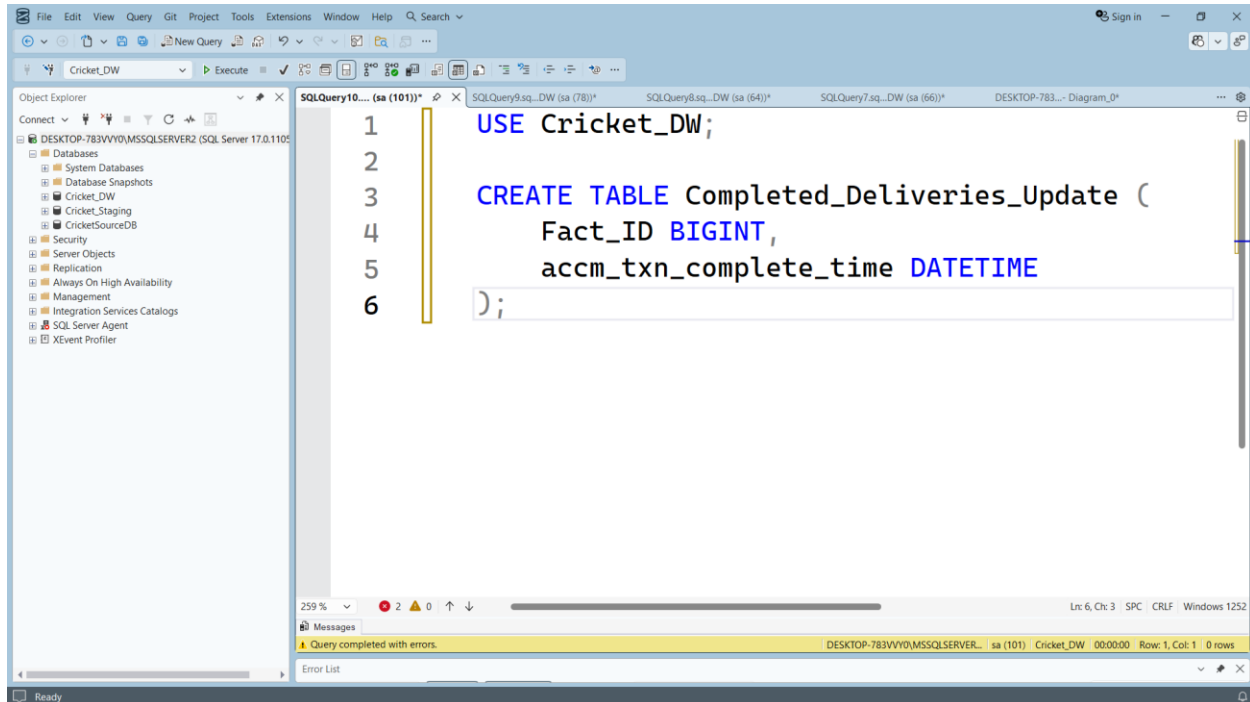


This ensures that every record in the fact table has a starting timestamp recorded.



Prepare a Separate Data Set (Update Table)

I created a separate staging table named Completed_Deliveries_Update to simulate the completion of specific delivery transactions.

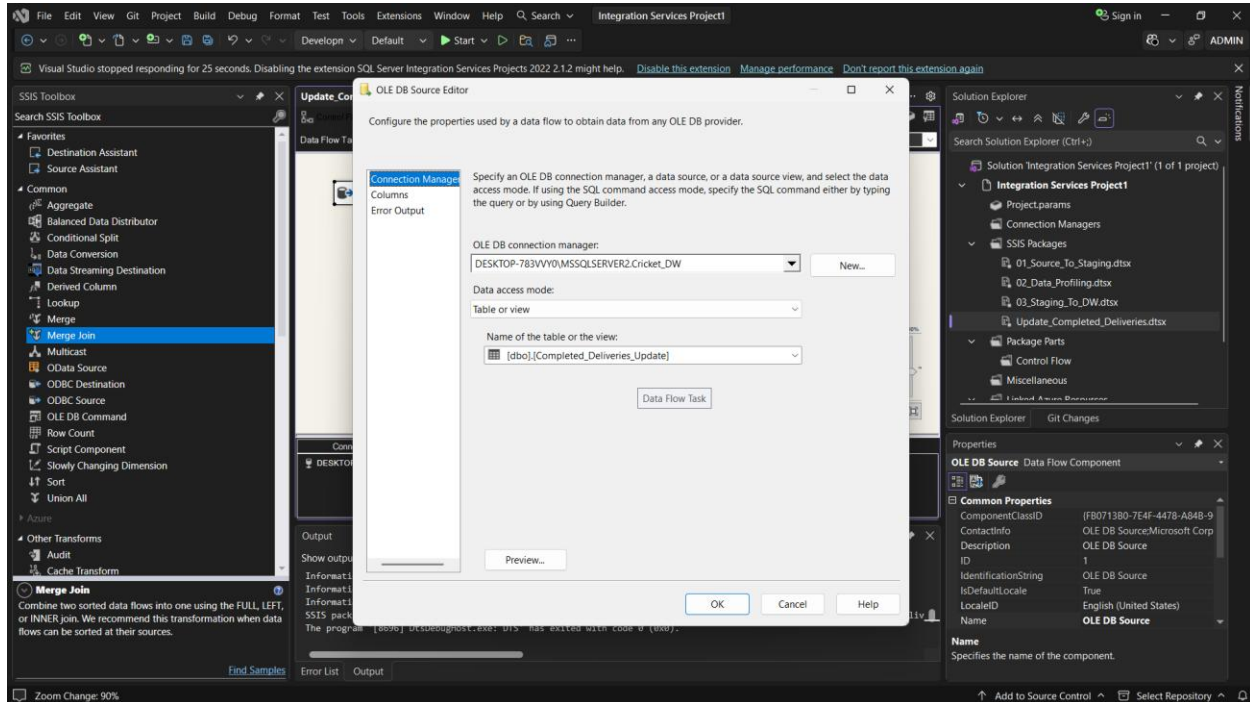


SSIS Package Development (The Update Pipeline)

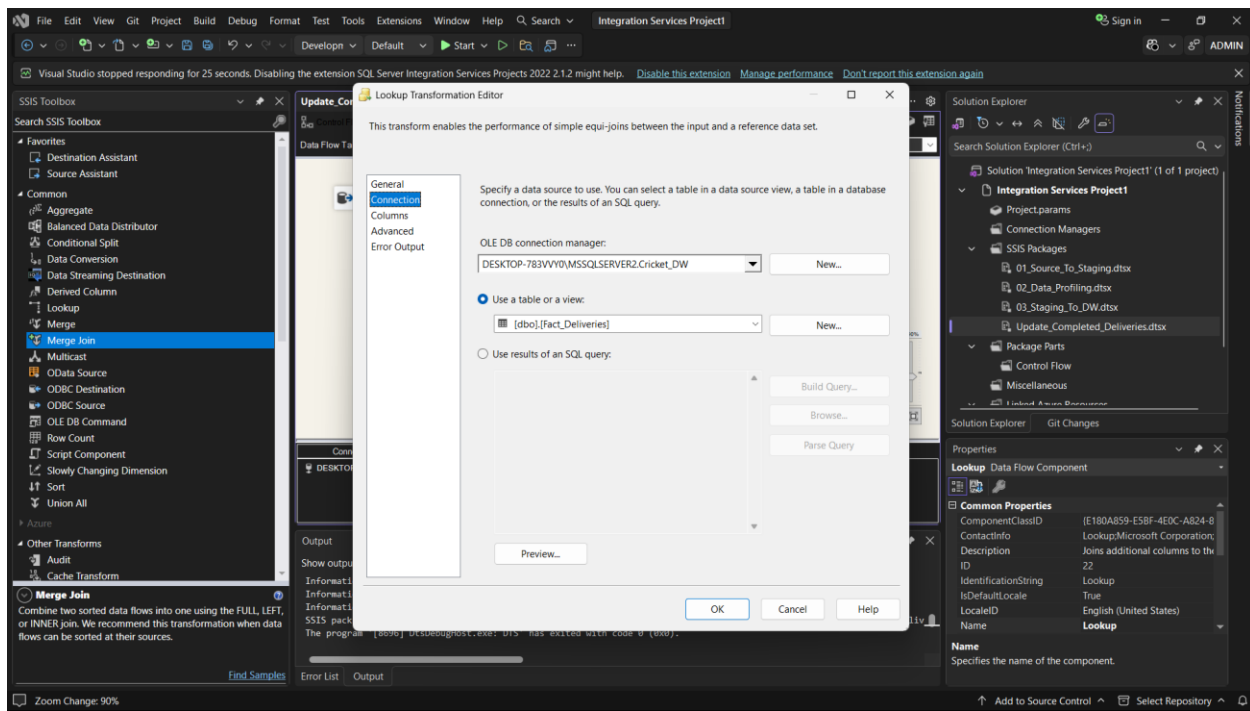
A dedicated SSIS package was developed to update the fact table. The flow includes:

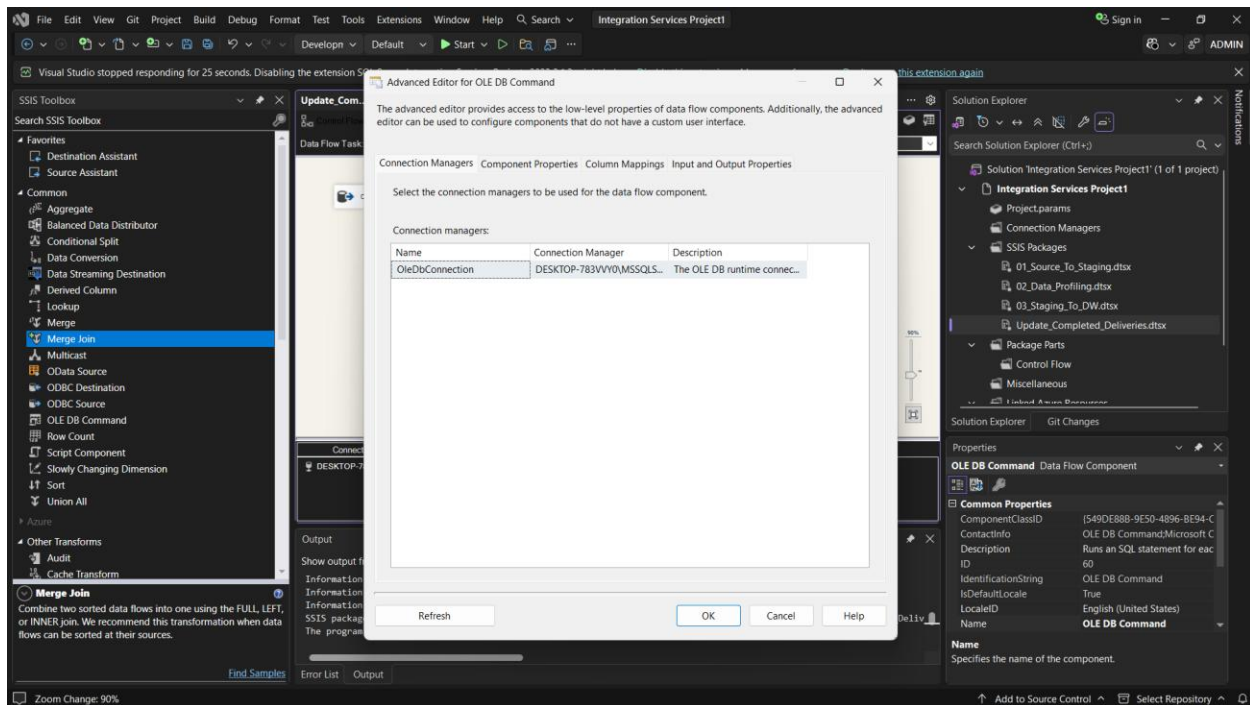
- **OLE DB Source:** Extracts data from the Completed_Deliveries_Update table.
- **Lookup Transformation:** Joins the source with the Fact_Deliveries table based on Fact_ID to retrieve existing record metadata.
- **Derived Column:** Used to calculate the txn_process_time_hours using the DATEDIFF function to avoid data type conversion errors.
- **OLE DB Command:** Executes the parameterized SQL UPDATE statement for each row in the data flow. It maps the transformed data (Completion Time, Calculated Hours, and Fact ID) to the destination table's columns to finalize the record update.

In Visual Studio, OLE DBSource

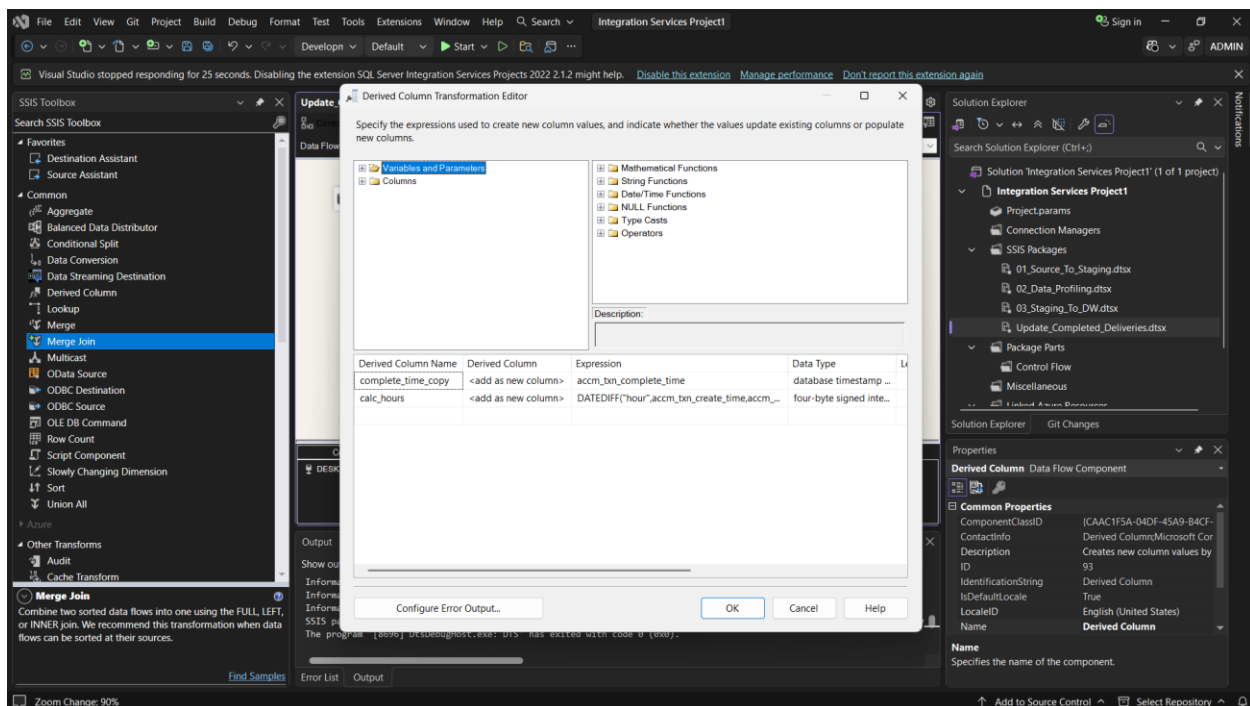


Here, Lookup Transformation,





Here, Derived Columns,



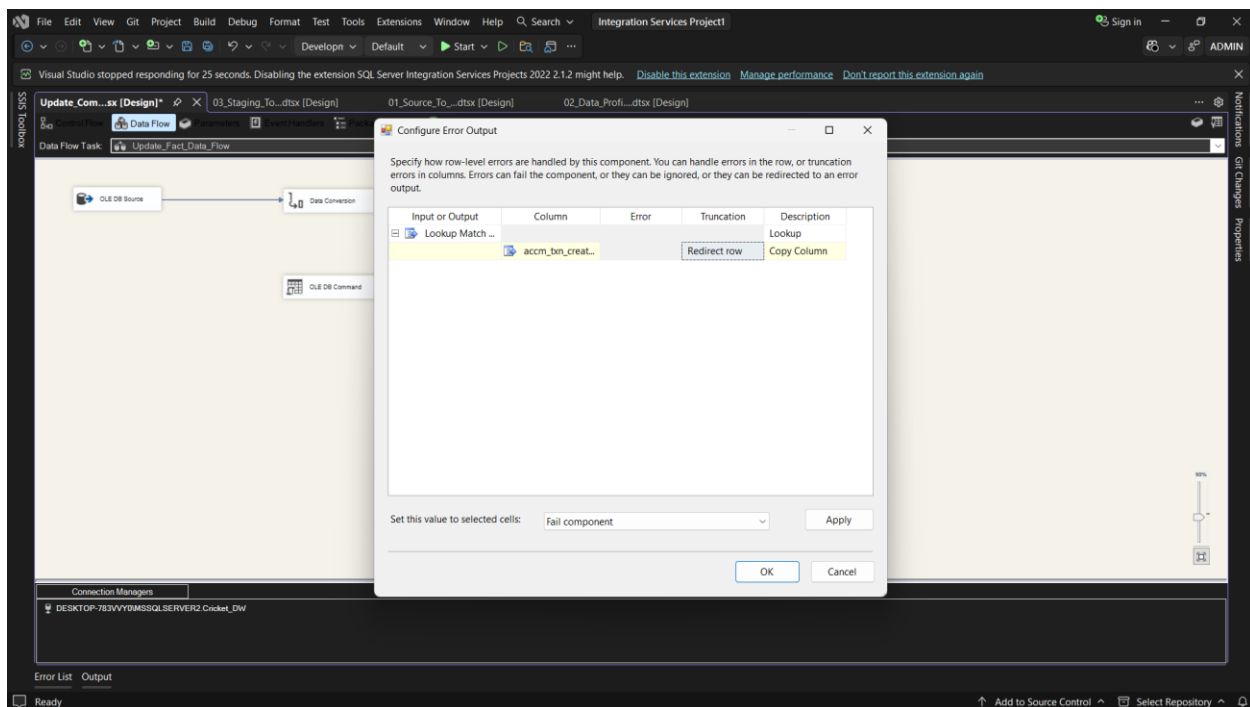
Handling Missing Transaction IDs (Fact_ID)

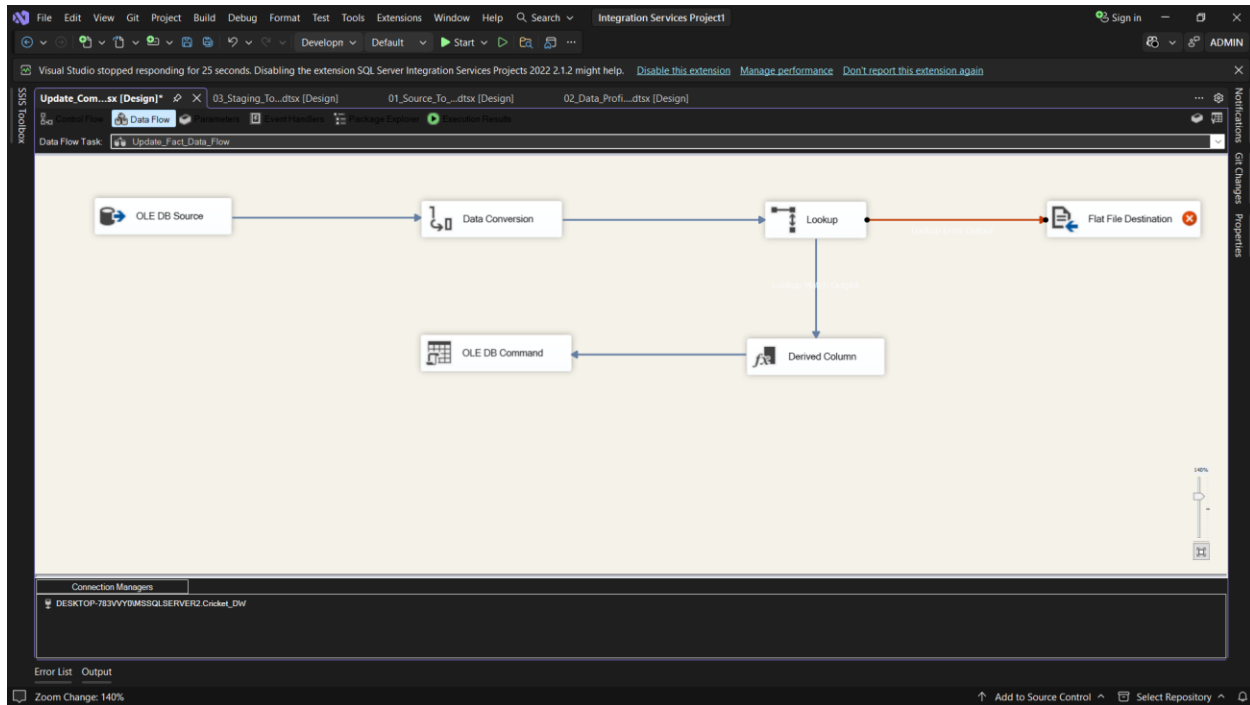
During the update process of the accumulating fact table, it is crucial to handle scenarios where an incoming transaction ID (Fact_ID) from the update dataset does not exist in the current Fact_Deliveries table.

To address this missing transaction ID scenario, the Lookup transformation in the update SSIS package was configured to handle mismatched records gracefully. Instead of failing the entire component, the **"No Match Output"** (error path) of the Lookup was configured to **"Redirect row"**.

This ensures that:

1. Only valid, existing records successfully pass through to the OLE DB Command to be updated with the accm_txn_complete_time and txn_process_time_hours.
2. Any missing or invalid transaction IDs are diverted from the main flow (e.g., to an error flat file), preventing the pipeline from crashing and maintaining the integrity of the accumulating snapshot process.





Update Query Used in OLE DB Command:

UPDATE Fact_Deliveries

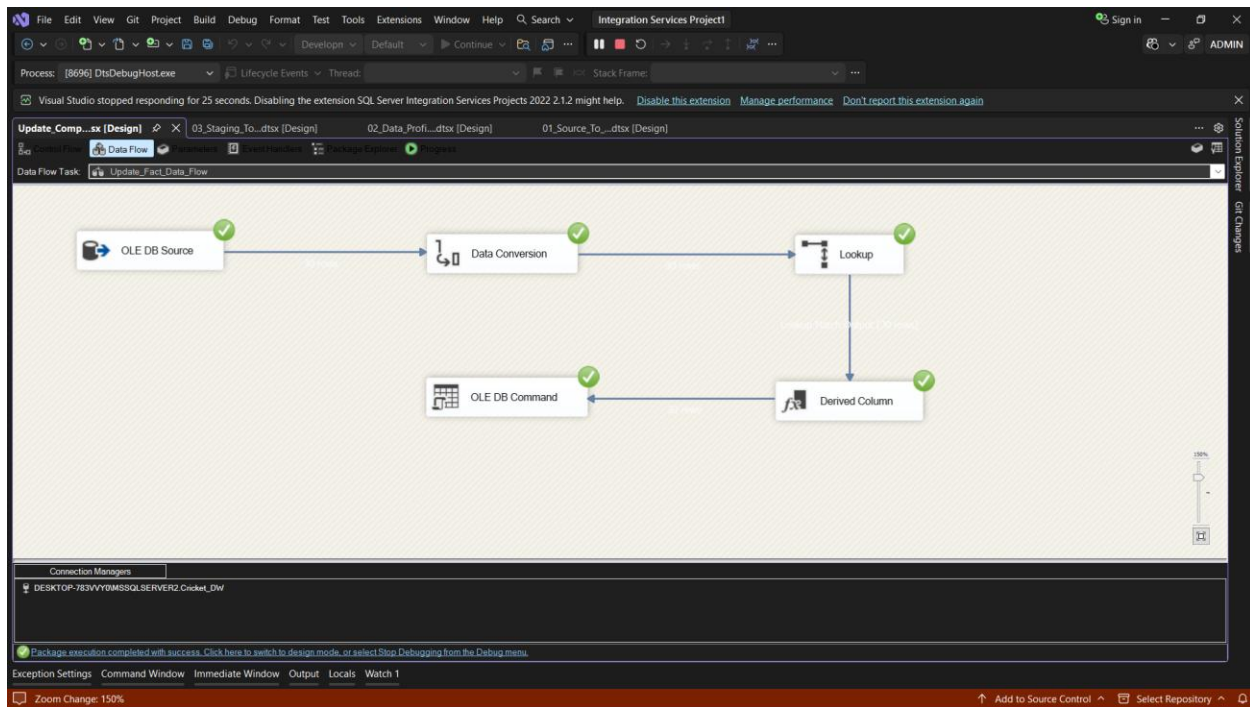
SET

accm_txn_complete_time = ?,

txn_process_time_hours = ?

WHERE Fact_ID = ?

The update query sets the accm_txn_complete_time and the pre-calculated txn_process_time_hours. This allows for accurate tracking of how long it took for each delivery to be fully processed.



Verify in SQL Server (Final Output)

After running the package, I verified the results in SQL Server Management Studio (SSMS) to ensure the records were correctly updated.

```
SELECT Fact_ID, accm_txn_create_time, accm_txn_complete_time,  
txn_process_time_hours  
FROM Fact_Deliveries  
WHERE accm_txn_complete_time IS NOT NULL;
```

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'DESKTOP-783VVY0\MSSQLSERVER2 (SQL Server 17.0.1105)'. The central pane shows a SQL query being executed in the 'Cricket_DW' database. The query is as follows:

```
1 USE Cricket_DW;
2
3 SELECT
4     Fact_ID,
5     accm_txn_create_time,
6     accm_txn_complete_time,
7     txn_process_time_hours
8 FROM Fact_Deliveries
9 WHERE accm_txn_complete_time IS NOT NULL;
```

The Results pane at the bottom shows the output of the query, displaying 30 rows of data. The columns are Fact_ID, accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours. The status bar at the bottom indicates 'Query executed successfully. DESKTOP-783VVY0\MSSQLSERVER... sa (101) Cricket_DW 00:00:00 Row: 1, Col: 1 30 rows'.

Fact_ID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
2	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
3	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
4	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
5	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
6	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
7	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
8	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
9	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
10	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
11	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
12	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
13	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
14	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
15	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48
16	2026-03-29 09:40:05.577	2026-03-31 09:40:05.577	48

The successfully updated table demonstrates that the accumulating snapshot logic is working as expected, capturing both the lifecycle timestamps and the duration of the process.